Speaker notes pentru prezentarea licenței

Second Brain — analiză tehnică a implementării existente

> **Repository:** `alexandru-strugariu15/licenta_v_1.0`, branch `v1.0`
>
> **Scop:** document de citit în timpul screen share-ului, cu termeni backend, fișiere și linii importante.
>
> **Verdict tehnic:** codul actual este un **frontend prototype** funcțional vizual; backend-ul descris în plan este următorul milestone, nu o funcționalitate deja livrată.

Generat la 3 noiembrie 2026 pe baza surselor din repository și a documentelor proiectului.

1. Rezumat de spus coordonatorului

„Am construit fundația unei aplicații de tip **Second Brain**, orientată spre notițe, editor block-based, desen pe canvas, task-uri, knowledge graph și Copilot. Am folosit **React** cu **TypeScript**, build-uit cu **Vite**, iar starea globală este gestionată cu **Zustand** și persistată local. Am separat aplicația în pagini și componente reutilizabile și am definit fluxurile de autentificare demo, onboarding, editor, căutare, task-uri, grafic și sincronizare.”

„Partea importantă de arhitectură pentru licență este documentată: următorul pas este să înlocuiesc datele mock cu un backend real bazat pe **Supabase/Postgres**, sincronizare **CRDT/Yjs** prin **Hocuspocus**, persistență offline și un API server pentru **RAG** și citări. În starea actuală nu pretind că aceste servicii sunt implementate.”

2. Ce există acum vs. ce este planificat

Suprafață	Implementat acum	Planificat pentru backend
Autentificare	Formular și utilizator demo în Zustand.	Supabase Auth, JWT, recovery și validare server-side.
Date	Workspace/notebook/page/task în memorie și localStorage.	Postgres/Supabase cu relații, roluri și workspace membership.
Editare	TipTap produce HTML local.	Y.Doc per pagină, snapshots și colaborare realtime.
Canvas	Stroke-uri și undo/redo local în React state.	Stroke-uri în Y.Map, awareness pentru live drawing și Y.UndoManager.
Copilot	Răspunsuri și recunoaștere simulate în UI.	`POST /ai/ask`, embeddings, top-k chunks, răspuns grounded și citări.
Sync/P2P	Dialog și peers mock.	WebRTC/y-webrtc cu signaling; BLE/Wi-Fi Direct rămân future work pentru browser.

> **Formulare sigură la prezentare:** spune „am implementat interfața și fluxul demonstrativ” pentru Auth/Copilot/Sync, nu „am implementat autentificarea reală, AI-ul sau sincronizarea distribuită”.

3. Tehnologiile folosite și rolul lor

- **TypeScript** — tipuri pentru User, Workspace, Notebook, Page, Task, Stroke și starea aplicației.
- **React 19** — componente, hooks și randare declarativă.
- **Vite 6** — development server și bundling pentru producție.
- **React Router** — rute pentru auth, onboarding, editor, graph și tasks.
- **Zustand + persist middleware** — store global și persistență locală în browser.
- **TipTap/ProseMirror** — editor rich-text extensibil, fundație pentru block editor.
- **HTML Canvas + perfect-freehand** — desen cu puncte, presiune și contururi naturale.
- **Lucide React** — iconițe consistente pentru toolbar și navigare.
- **pnpm workspace** — structură de monorepo pregătită pentru web/server/packages.

Backend terminology: un **endpoint** este o adresă HTTP a serverului; un **JWT** este tokenul de autentificare; o **tranzacție** este o modificare atomică; un **CRDT** permite convergența modificărilor concurente fără conflict central; un **snapshot** este starea serializată a unui document; **RAG** înseamnă retrieval-augmented generation, adică răspuns generat pe baza unor fragmente recuperate din notițe; un **embedding** este un vector folosit pentru căutare semantică.

4. Fișierele și codul de prezentat

apps/web/src/App.tsx

Routerul și gardul de acces al aplicației. Decide dacă utilizatorul merge la autentificare, onboarding sau în shell-ul principal. În termeni de backend, acesta este punctul unde UI-ul aplică starea de sesiune, dar sesiunea este doar locală în prototip.

Linii importante: 1-31

```
 1 | import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom'
 2 | import { AppShell } from './components/AppShell'
 3 | import { AuthPage } from './pages/AuthPage'
 4 | import { OnboardingPage } from './pages/OnboardingPage'
 5 | import { useAppStore } from './store/appStore'
 6 | import './index.css'
 7 |
 8 | function App() {
 9 |   const { isAuthenticated, hasCompletedOnboarding } = useAppStore()
10 |
11 |   return (
12 |     <BrowserRouter>
13 |       <Routes>
14 |         <Route path="/auth" element={<AuthPage />} />
15 |         <Route path="/onboarding" element={<OnboardingPage />} />
16 |         <Route
17 |           path="/*"
18 |           element={
19 |             !isAuthenticated ? (
20 |               <Navigate to="/auth" replace />
21 |             ) : !hasCompletedOnboarding ? (
22 |               <Navigate to="/onboarding" replace />
23 |             ) : (
24 |               <AppShell />
25 |             )
26 |           }
27 |         />
28 |       </Routes>
29 |     </BrowserRouter>
30 |   )
31 | }
```

apps/web/src/store/appStore.ts

Store global Zustand. Conține tipurile de domeniu (User, Workspace, Notebook, Page), starea de autentificare, navigare, sincronizare și acțiunile de modificare. Middleware-ul **persist** salvează o parte din stare în localStorage; nu există încă API sau bază de date.

Linii importante: 1-44

```
 1 | import { create } from 'zustand'
 2 | import { persist } from 'zustand/middleware'
 3 |
 4 | export type SyncStatus = 'synced' | 'syncing' | 'offline' | 'error'
 5 | export type Theme = 'light' | 'dark'
 6 |
 7 | export interface Workspace {
 8 |   id: string
 9 |   name: string
10 |   kind: 'personal' | 'university' | 'project'
11 | }
12 |
13 | export interface Notebook {
14 |   id: string
15 |   workspaceId: string
16 |   name: string
17 |   emoji: string
18 |   pages: Page[]
19 | }
20 |
21 | export interface Page {
22 |   id: string
23 |   notebookId: string
24 |   title: string
25 |   icon: string
26 |   updatedAt: string
27 | }
28 |
29 | export interface User {
30 |   id: string
31 |   name: string
32 |   email: string
33 |   initials: string
34 | }
35 |
36 | interface AppState {
37 |   // Auth
38 |   isAuthenticated: boolean
39 |   hasCompletedOnboarding: boolean
40 |   user: User | null
41 |
42 |   // Theme
```

```
  43 |   theme: Theme
  44 |
```

Linii importante: 46-75

```
  46 |   currentWorkspace: Workspace | null
  47 |   currentNotebookId: string | null
  48 |   currentPageId: string | null
  49 |   sidebarCollapsed: boolean
  50 |   copilotOpen: boolean
  51 |
  52 |   // Data
  53 |   workspaces: Workspace[]
  54 |   notebooks: Notebook[]
  55 |
  56 |   // Sync
  57 |   syncStatus: SyncStatus
  58 |   offlineChanges: number
  59 |
  60 |   // UI
  61 |   searchOpen: boolean
  62 |
  63 |   // Actions
  64 |   login: (user: User) => void
  65 |   logout: () => void
  66 |   completeOnboarding: (workspace: Workspace) => void
  67 |   setTheme: (theme: Theme) => void
  68 |   toggleSidebar: () => void
  69 |   toggleCopilot: () => void
  70 |   setCurrentPage: (pageId: string | null) => void
  71 |   setCurrentNotebook: (notebookId: string | null) => void
  72 |   setSyncStatus: (status: SyncStatus) => void
  73 |   setSearchOpen: (open: boolean) => void
  74 |   addNotebook: (notebook: Notebook) => void
  75 |   addPage: (notebookId: string, page: Page) => void
```

Linii importante: 94-151

```
  94 |     workspaceId: 'ws-1',
  95 |     name: 'Sisteme Automate',
  96 |     emoji: '🤖',
  97 |     pages: [
  98 |       { id: 'pg-1', notebookId: 'nb-1', title: 'Feedback Stability', icon: '📊', updatedAt: new
Date().toISOString() },
  99 |       { id: 'pg-2', notebookId: 'nb-1', title: 'Bode Plots', icon: '📈', updatedAt: new Date().toISOString()
},
 100 |       { id: 'pg-3', notebookId: 'nb-1', title: 'Root Locus', icon: '✏️', updatedAt: new Date().toISOString()
},
 101 |     ],
 102 |   },
 103 |   {
```

```
104 |     id: 'nb-2',
105 |     workspaceId: 'ws-1',
106 |     name: 'Programare Avansată',
107 |     emoji: '💻',
108 |     pages: [
109 |       { id: 'pg-4', notebookId: 'nb-2', title: 'Design Patterns', icon: '🏗️', updatedAt: new
Date().toISOString() },
110 |       { id: 'pg-5', notebookId: 'nb-2', title: 'CRDT & Sync', icon: '🔄', updatedAt: new
Date().toISOString() },
111 |     ],
112 |   },
113 | ]
114 |
115 | export const useAppStore = create<AppState>()(
116 |   persist(
117 |     (set) => ({
118 |       isAuthenticated: false,
119 |       hasCompletedOnboarding: false,
120 |       user: null,
121 |       theme: 'light',
122 |       currentWorkspace: null,
123 |       currentNotebookId: 'nb-1',
124 |       currentPageId: 'pg-1',
125 |       sidebarCollapsed: false,
126 |       copilotOpen: false,
127 |       workspaces: [],
128 |       notebooks: DEMO_NOTEBOOKS,
129 |       syncStatus: 'synced',
130 |       offlineChanges: 0,
131 |       searchOpen: false,
132 |
133 |       login: (user) =>
134 |         set({
135 |           isAuthenticated: true,
136 |           user,
137 |           currentWorkspace: DEMO_WORKSPACE,
138 |           workspaces: [DEMO_WORKSPACE],
139 |           notebooks: DEMO_NOTEBOOKS,
140 |         }),
141 |
142 |       logout: () =>
143 |         set({
144 |           isAuthenticated: false,
145 |           hasCompletedOnboarding: false,
146 |           user: null,
147 |           currentWorkspace: null,
148 |         }),
149 |
```

```
150 |       completeOnboarding: (workspace) =>
151 |         set({
```

Linii importante: 153-191

```
153 |           currentWorkspace: workspace,
154 |           workspaces: [workspace],
155 |         }),
156 |
157 |       setTheme: (theme) => {
158 |         document.documentElement.setAttribute('data-theme', theme)
159 |         set({ theme })
160 |       },
161 |
162 |       toggleSidebar: () => set((s) => ({ sidebarCollapsed: !s.sidebarCollapsed })),
163 |       toggleCopilot: () => set((s) => ({ copilotOpen: !s.copilotOpen })),
164 |
165 |       setCurrentPage: (pageId) => set({ currentPageId: pageId }),
166 |       setCurrentNotebook: (notebookId) => set({ currentNotebookId: notebookId }),
167 |       setSyncStatus: (syncStatus) => set({ syncStatus }),
168 |       setSearchOpen: (searchOpen) => set({ searchOpen }),
169 |       addNotebook: (notebook) => set((s) => ({ notebooks: [...s.notebooks, notebook] })),
170 |       addPage: (notebookId, page) =>
171 |         set((s) => ({
172 |           notebooks: s.notebooks.map((nb) =>
173 |             nb.id === notebookId ? { ...nb, pages: [...nb.pages, page] } : nb
174 |           ),
175 |         })),
176 |     }),
177 |     {
178 |       name: 'second-brain-app',
179 |       partialize: (s) => ({
180 |         isAuthenticated: s.isAuthenticated,
181 |         hasCompletedOnboarding: s.hasCompletedOnboarding,
182 |         user: s.user,
183 |         theme: s.theme,
184 |         currentWorkspace: s.currentWorkspace,
185 |         currentPageId: s.currentPageId,
186 |         currentNotebookId: s.currentNotebookId,
187 |         workspaces: s.workspaces,
188 |         notebooks: s.notebooks,
189 |       }),
190 |     }
191 |   )
```

apps/web/src/components/AppShell.tsx

Containerul aplicației autentificate. Montează sidebar-ul, rutele editor/graph/tasks și panourile condiționale. Shortcut-ul Ctrl/Cmd+K deschide căutarea globală.

Linii importante: 1-47

```
 1 | import { Routes, Route } from 'react-router-dom'
 2 | import { Sidebar } from './Sidebar'
 3 | import { EditorPage } from '../pages/EditorPage'
 4 | import { KnowledgeGraphPage } from '../pages/KnowledgeGraphPage'
 5 | import { TasksPage } from '../pages/TasksPage'
 6 | import { CopilotPanel } from './CopilotPanel'
 7 | import { SearchDialog } from './SearchDialog'
 8 | import { useAppStore } from '../store/appStore'
 9 | import { useEffect } from 'react'
10 |
11 | export function AppShell() {
12 |   const { theme, searchOpen, copilotOpen } = useAppStore()
13 |
14 |   useEffect(() => {
15 |     document.documentElement.setAttribute('data-theme', theme)
16 |   }, [theme])
17 |
18 |   // Global keyboard shortcuts
19 |   useEffect(() => {
20 |     const handler = (e: KeyboardEvent) => {
21 |       if ((e.metaKey || e.ctrlKey) && e.key === 'k') {
22 |         e.preventDefault()
23 |         useAppStore.getState().setSearchOpen(true)
24 |       }
25 |     }
26 |     window.addEventListener('keydown', handler)
27 |     return () => window.removeEventListener('keydown', handler)
28 |   }, [])
29 |
30 |   return (
31 |     <div className="app-shell">
32 |       <Sidebar />
33 |
34 |       <div className="main-content">
35 |         <Routes>
36 |           <Route path="/" element={<EditorPage />} />
37 |           <Route path="/page/:pageId" element={<EditorPage />} />
38 |           <Route path="/graph" element={<KnowledgeGraphPage />} />
39 |           <Route path="/tasks" element={<TasksPage />} />
40 |         </Routes>
41 |       </div>
42 |
43 |       {copilotOpen && <CopilotPanel />}
44 |       {searchOpen && <SearchDialog />}
```

```
45 |     </div>
46 |   )
47 | }
```

apps/web/src/pages/AuthPage.tsx

Flux demonstrativ de autentificare. Are formulare de login/register și creează un utilizator local; nu validează parole în backend și nu folosește Supabase Auth.

Linii importante: 1-32

```
 1 | import { useState } from 'react'
 2 | import { Sparkles, Eye, EyeOff } from 'lucide-react'
 3 | import { useAppStore } from '../store/appStore'
 4 | import { useNavigate } from 'react-router-dom'
 5 |
 6 | export function AuthPage() {
 7 |   const { login } = useAppStore()
 8 |   const navigate = useNavigate()
 9 |
10 |   const [mode, setMode] = useState<'login' | 'register'>('login')
11 |   const [name, setName] = useState('')
12 |   const [email, setEmail] = useState('')
13 |   const [password, setPassword] = useState('')
14 |   const [showPassword, setShowPassword] = useState(false)
15 |   const [loading, setLoading] = useState(false)
16 |   const [error, setError] = useState('')
17 |
18 |   async function handleSubmit(e: React.FormEvent) {
19 |     e.preventDefault()
20 |     setError('')
21 |     setLoading(true)
22 |
23 |     // Simulate auth
24 |     await new Promise((r) => setTimeout(r, 800))
25 |
26 |     const user = {
27 |       id: 'user-1',
28 |       name: name || 'Alexandru Strugariu',
29 |       email: email || 'alex@example.com',
30 |       initials: (name || 'Alexandru Strugariu')
31 |         .split(' ')
32 |         .map((n) => n[0])
```

Linii importante: 43-58

```
43 |   function handleDemoLogin() {
44 |     login({
45 |       id: 'user-demo',
46 |       name: 'Alexandru Strugariu',
47 |       email: 'alex@unstpb.ro',
48 |       initials: 'AS',
49 |     })
50 |     useAppStore.getState().completeOnboarding({
51 |       id: 'ws-demo',
52 |       name: 'Licență UNSTPB',
```

```
  53 |       kind: 'university',
  54 |     })
  55 |     navigate('/')
  56 |   }
  57 |
  58 |   return (
```

Linii importante: 160-185

```
 160 |           <div style={{ flex: 1, height: 1, background: 'var(--color-border-subtle)' }} />
 161 |           <span style={{ fontSize: 12, color: 'var(--color-text-tertiary)' }}>or</span>
 162 |           <div style={{ flex: 1, height: 1, background: 'var(--color-border-subtle)' }} />
 163 |         </div>
 164 |
 165 |         {/* Demo login */}
 166 |         <button
 167 |           className="btn btn-secondary btn-lg"
 168 |           style={{ width: '100%' }}
 169 |           onClick={handleDemoLogin}
 170 |         >
 171 |           <Sparkles size={15} />
 172 |           Try demo workspace
 173 |         </button>
 174 |
 175 |         {/* Toggle mode */}
 176 |         <p style={{ textAlign: 'center', marginTop: 20, fontSize: 13, color: 'var(--color-text-secondary)'
}}>
 177 |           {mode === 'login' ? "Don't have an account? " : 'Already have an account? '}
 178 |           <button
 179 |             type="button"
 180 |             style={{ color: 'var(--color-accent)', fontWeight: 600, background: 'none', border: 'none',
cursor: 'pointer', font: 'inherit' }}
 181 |             onClick={() => { setMode(mode === 'login' ? 'register' : 'login'); setError('') }}
 182 |           >
 183 |             {mode === 'login' ? 'Sign up' : 'Sign in'}
 184 |           </button>
 185 |         </p>
```

apps/web/src/pages/OnboardingPage.tsx

Onboarding pentru workspace. Utilizatorul alege scopul workspace-ului (personal/university/project), apoi se memorează un workspace local.

Linii importante: 1-35

```
  1 | import { useState } from 'react'
  2 | import { Sparkles, ArrowRight, BookOpen, GraduationCap, Briefcase } from 'lucide-react'
  3 | import { useAppStore, type Workspace } from '../store/appStore'
  4 | import { useNavigate } from 'react-router-dom'
  5 |
  6 | type WorkspaceKind = 'personal' | 'university' | 'project'
  7 |
  8 | const WORKSPACE_OPTIONS: { kind: WorkspaceKind; icon: React.ReactNode; label: string; description: string }[]
= [
  9 |   { kind: 'personal', icon: <BookOpen size={24} />, label: 'Personal', description: 'Journaling, ideas, and
personal knowledge' },
 10 |   { kind: 'university', icon: <GraduationCap size={24} />, label: 'University', description: 'Lecture notes,
assignments, and research' },
 11 |   { kind: 'project', icon: <Briefcase size={24} />, label: 'Project', description: 'Team notes, sprints, and
documentation' },
 12 | ]
 13 |
 14 | const STEPS = ['Purpose', 'Workspace', 'Done']
 15 |
 16 | export function OnboardingPage() {
 17 |   const { completeOnboarding } = useAppStore()
 18 |   const navigate = useNavigate()
 19 |
 20 |   const [step, setStep] = useState(0)
 21 |   const [selectedKind, setSelectedKind] = useState<WorkspaceKind>('university')
 22 |   const [workspaceName, setWorkspaceName] = useState('')
 23 |
 24 |   function handleFinish() {
 25 |     const workspace: Workspace = {
 26 |       id: crypto.randomUUID(),
 27 |       name: workspaceName || (selectedKind === 'university' ? 'Licență UNSTPB' : selectedKind === 'project' ?
'My Project' : 'My Brain'),
 28 |       kind: selectedKind,
 29 |     }
 30 |     completeOnboarding(workspace)
 31 |     navigate('/')
 32 |   }
 33 |
 34 |   return (
 35 |     <div className="auth-page" style={{ alignItems: 'center' }}>
```

Linii importante: 116-188

```
116 |
117 |             <button
```

```
118 |               className="btn btn-primary btn-lg"
119 |               style={{ width: '100%', marginTop: 24 }}
120 |               onClick={() => setStep(1)}
121 |             >
122 |               Continue <ArrowRight size={16} />
123 |             </button>
124 |           </div>
125 |         )}
126 |
127 |         {/* Step 1: Name workspace */}
128 |         {step === 1 && (
129 |           <div className="auth-card">
130 |             <h2 style={{ fontSize: 22, fontWeight: 700, color: 'var(--color-text-primary)', marginBottom: 6
}}>
131 |               Name your workspace
132 |             </h2>
133 |             <p style={{ fontSize: 14, color: 'var(--color-text-secondary)', marginBottom: 24 }}>
134 |               Give your {selectedKind} workspace a name.
135 |             </p>
136 |
137 |             <div className="form-group">
138 |               <label className="form-label" htmlFor="ws-name">Workspace name</label>
139 |               <input
140 |                 id="ws-name"
141 |                 type="text"
142 |                 className="form-input"
143 |                 placeholder={selectedKind === 'university' ? 'Licență UNSTPB' : selectedKind === 'project' ?
'My Project' : 'My Second Brain'}
144 |                 value={workspaceName}
145 |                 onChange={(e) => setWorkspaceName(e.target.value)}
146 |                 autoFocus
147 |               />
148 |             </div>
149 |
150 |             <div style={{ display: 'flex', gap: 12, marginTop: 24 }}>
151 |               <button className="btn btn-secondary btn-lg" style={{ flex: 1 }} onClick={() => setStep(0)}>
152 |                 Back
153 |               </button>
154 |               <button className="btn btn-primary btn-lg" style={{ flex: 2 }} onClick={() => setStep(2)}>
155 |                 Continue <ArrowRight size={16} />
156 |               </button>
157 |             </div>
158 |           </div>
159 |         )}
160 |
161 |         {/* Step 2: Done */}
162 |         {step === 2 && (
163 |           <div className="auth-card" style={{ textAlign: 'center' }}>
164 |             <div style={{ fontSize: 56, marginBottom: 16 }}>🎉</div>
165 |             <h2 style={{ fontSize: 22, fontWeight: 700, color: 'var(--color-text-primary)', marginBottom: 8
```

```
}}>
 166 |               You're all set!
 167 |             </h2>
 168 |             <p style={{ fontSize: 14, color: 'var(--color-text-secondary)', marginBottom: 8 }}>
 169 |               Your workspace <strong>{workspaceName || 'Licență UNSTPB'}</strong> is ready.
 170 |             </p>
 171 |             <p style={{ fontSize: 13, color: 'var(--color-text-tertiary)', marginBottom: 32, lineHeight: 1.6
}}>
 172 |               Start taking notes with block-based editing, drawing with ink, and asking your AI co-pilot
questions about your notes.
 173 |             </p>
 174 |
 175 |             <div style={{ display: 'flex', flexDirection: 'column', gap: 10, marginBottom: 24, textAlign:
'left' }}>
 176 |               {[
 177 |                 '✏️  Press / to insert blocks',
 178 |                 '🖊️  Toggle a drawing canvas from any page',
 179 |                 '✧  Ask your AI Co-pilot about your notes',
 180 |                 '⌘K  Quick search across all pages',
 181 |               ].map((hint) => (
 182 |                 <div key={hint} style={{ fontSize: 13, color: 'var(--color-text-secondary)', display: 'flex',
gap: 8 }}>
 183 |                   {hint}
 184 |                 </div>
 185 |               ))}
 186 |             </div>
 187 |
 188 |             <button className="btn btn-primary btn-lg" style={{ width: '100%' }} onClick={handleFinish}>
```

apps/web/src/pages/EditorPage.tsx

Pagina principală de lucru. Leagă pagina curentă din store de editor, schimbarea titlului/iconiței, statusul de sync, Copilot și CanvasBlock. Unele butoane sunt încă placeholder (`onClick={() => {}}`).

Linii importante: 1-39

```
1 | import { useState, useEffect } from 'react'
2 | import {
3 |   Share2, Sparkles, Menu, Wifi, WifiOff, Loader,
4 |   MoreHorizontal, Pencil
5 | } from 'lucide-react'
6 | import { RichEditor } from '../components/RichEditor'
7 | import { CanvasBlock } from '../components/CanvasBlock'
8 | import { useAppStore } from '../store/appStore'
9 |
10 | const PAGE_ICONS = ['📄', '📊', '📈', '✏️', '🏗️', '🔄', '💡', '📚', '🎯', '⚙️', '🔬', '🧮']
11 |
12 | export function EditorPage() {
13 |   const {
14 |     notebooks, currentPageId, currentNotebookId,
15 |     toggleCopilot, copilotOpen, toggleSidebar, sidebarCollapsed,
16 |     syncStatus
17 |   } = useAppStore()
18 |
19 |   const currentNotebook = notebooks.find((nb) => nb.id === currentNotebookId)
20 |   const currentPage = currentNotebook?.pages.find((p) => p.id === currentPageId)
21 |
22 |   const [title, setTitle] = useState(currentPage?.title ?? 'Untitled')
23 |   const [pageIcon, setPageIcon] = useState(currentPage?.icon ?? '📄')
24 |   const [showIconPicker, setShowIconPicker] = useState(false)
25 |   const [showCanvas, setShowCanvas] = useState(false)
26 |
27 |   useEffect(() => {
28 |     setTitle(currentPage?.title ?? 'Untitled')
29 |     setPageIcon(currentPage?.icon ?? '📄')
30 |   }, [currentPageId])
31 |
32 |   const SyncIcon = syncStatus === 'syncing' ? Loader : syncStatus === 'offline' ? WifiOff : Wifi
33 |   const syncColor = {
34 |     synced: '#067647', syncing: '#B54708', offline: 'var(--color-text-tertiary)', error: '#B42318',
35 |   }[syncStatus]
36 |
37 |   return (
38 |     <div className="editor-container">
39 |       {/* Top bar */}
```

Linii importante: 100-131

```
100 |           {/* Sync status */}
101 |           <div
102 |             className="sync-badge"
```

```
103 |             style={{ color: syncColor }}
104 |           >
105 |             <SyncIcon
106 |               size={11}
107 |               className={syncStatus === 'syncing' ? 'animate-spin' : ''}
108 |             />
109 |             <span style={{ color: syncColor }}>
110 |               {syncStatus === 'synced' ? 'Page synced' : syncStatus === 'syncing' ? 'Saving…' : syncStatus
=== 'offline' ? 'Offline' : 'Error'}
111 |             </span>
112 |           </div>
113 |
114 |           <button className="btn btn-secondary btn-sm" onClick={() => {}}>
115 |             <Share2 size={13} />
116 |             Share
117 |           </button>
118 |
119 |           <button
120 |             className={`btn btn-sm ${copilotOpen ? 'btn-primary' : 'btn-secondary'}`}
121 |             onClick={toggleCopilot}
122 |           >
123 |             <Sparkles size={13} />
124 |             AI Co-pilot
125 |           </button>
126 |
127 |           <button className="btn btn-icon btn-ghost" onClick={() => {}} aria-label="More options">
128 |             <MoreHorizontal size={16} />
129 |           </button>
130 |         </div>
131 |       </header>
```

Linii importante: 145-195

```
145 |                 {pageIcon}
146 |               </button>
147 |
148 |               {showIconPicker && (
149 |                 <div
150 |                   style={{
151 |                     position: 'absolute', top: '100%', left: 0, zIndex: 50,
152 |                     background: 'var(--color-bg-page)',
153 |                     border: '1px solid var(--color-border-default)',
154 |                     borderRadius: 10, boxShadow: 'var(--shadow-lg)',
155 |                     padding: 8, display: 'grid', gridTemplateColumns: 'repeat(6, 1fr)', gap: 4,
156 |                   }}
157 |                 >
158 |                   {PAGE_ICONS.map((icon) => (
159 |                     <button
160 |                       key={icon}
161 |                       onClick={() => { setPageIcon(icon); setShowIconPicker(false) }}
```

```
162 |                       style={{
163 |                         width: 36, height: 36, fontSize: 20, border: 'none', background: 'transparent',
164 |                         borderRadius: 6, cursor: 'pointer', transition: 'background 0.1s',
165 |                         display: 'flex', alignItems: 'center', justifyContent: 'center',
166 |                       }}
167 |                       onMouseEnter={(e) => { (e.currentTarget as HTMLButtonElement).style.background = 'var(-
-color-bg-hover)' }}
168 |                       onMouseLeave={(e) => { (e.currentTarget as HTMLButtonElement).style.background =
'transparent' }}
169 |                     >
170 |                       {icon}
171 |                     </button>
172 |                   ))}
173 |                 </div>
174 |               )}
175 |             </div>
176 |
177 |             {/* Optional page decoration */}
178 |             <div style={{ display: 'flex', gap: 14, marginBottom: 8 }}>
179 |               <button
180 |                 className="btn btn-ghost btn-xs"
181 |                 style={{ fontSize: 12, color: 'var(--color-text-tertiary)', padding: 0 }}
182 |                 onClick={() => setShowIconPicker((v) => !v)}
183 |               >
184 |                 😀 Add icon
185 |               </button>
186 |               <button
187 |                 className="btn btn-ghost btn-xs"
188 |                 style={{ fontSize: 12, color: 'var(--color-text-tertiary)', padding: 0 }}
189 |                 onClick={() => {}}
190 |               >
191 |                 🖼️ Add cover
192 |               </button>
193 |             </div>
194 |
195 |             {/* Title */}
```

Linii importante: 244-280

```
244 |                 lineHeight: 1.6,
245 |                 color: 'var(--color-text-secondary)',
246 |                 marginTop: 14,
247 |                 marginBottom: 4,
248 |               }}
249 |             >
250 |               How do we know a feedback system will settle rather than oscillate or diverge? Start with the
closed-loop poles, then check the stability margins.
251 |             </p>
252 |           </header>
253 |
```

```
254 |           {/* Toggle canvas */}
255 |           <div style={{ marginBottom: 16, display: 'flex', gap: 8 }}>
256 |             <button
257 |               className={`btn btn-sm ${showCanvas ? 'btn-primary' : 'btn-secondary'}`}
258 |               onClick={() => setShowCanvas((v) => !v)}
259 |             >
260 |               <Pencil size={13} />
261 |               {showCanvas ? 'Hide drawing' : 'Add drawing'}
262 |             </button>
263 |           </div>
264 |
265 |           {/* Canvas Block */}
266 |           {showCanvas && (
267 |             <div style={{ marginBottom: 24 }}>
268 |               <CanvasBlock />
269 |             </div>
270 |           )}
271 |
272 |           {/* Rich text editor */}
273 |           <RichEditor
274 |             placeholder="Write something, or press '/' for commands…"
275 |           />
276 |         </article>
277 |       </main>
278 |     </div>
279 |   )
280 | }
```

apps/web/src/components/RichEditor.tsx

Editor rich-text TipTap/ProseMirror. Configurează extensii pentru titluri, liste, code block, tabel, link, imagine, highlight și formatare. `onChange` produce HTML local; nu salvează încă în server/Yjs.

Linii importante: 1-49

```
  1 | import { useEditor, EditorContent, BubbleMenu } from '@tiptap/react'
  2 | import StarterKit from '@tiptap/starter-kit'
  3 | import Placeholder from '@tiptap/extension-placeholder'
  4 | import TaskList from '@tiptap/extension-task-list'
  5 | import TaskItem from '@tiptap/extension-task-item'
  6 | import Underline from '@tiptap/extension-underline'
  7 | import Highlight from '@tiptap/extension-highlight'
  8 | import Link from '@tiptap/extension-link'
  9 | import { SlashMenu } from './SlashMenu'
 10 | import {
 11 |   Bold, Italic, Underline as UnderlineIcon, Strikethrough,
 12 |   Code, Highlighter, Link as LinkIcon
 13 | } from 'lucide-react'
 14 | import { useRef, useCallback } from 'react'
 15 |
 16 | interface RichEditorProps {
 17 |   content?: string
 18 |   onChange?: (html: string) => void
 19 |   placeholder?: string
 20 | }
 21 |
 22 | export function RichEditor({ content, onChange, placeholder = 'Write something, or press \'/\' for
commands…' }: RichEditorProps) {
 23 |   const editorRef = useRef<HTMLDivElement>(null)
 24 |
 25 |   const editor = useEditor({
 26 |     extensions: [
 27 |       StarterKit.configure({
 28 |         heading: { levels: [1, 2, 3] },
 29 |         codeBlock: { languageClassPrefix: 'language-' },
 30 |       }),
 31 |       Placeholder.configure({ placeholder }),
 32 |       TaskList,
 33 |       TaskItem.configure({ nested: true }),
 34 |       Underline,
 35 |       Highlight.configure({ multicolor: false }),
 36 |       Link.configure({ openOnClick: true, autolink: true }),
 37 |     ],
 38 |     content: content || defaultContent,
 39 |     editorProps: {
 40 |       attributes: { class: 'tiptap-editor', spellcheck: 'true' },
 41 |     },
 42 |     onUpdate: ({ editor }) => {
 43 |       onChange?.(editor.getHTML())
```

```
44 |     },
45 |   })
46 |
47 |   // Slash command handler
48 |   const handleKeyDown = useCallback((_e: React.KeyboardEvent) => {
49 |     // SlashMenu handles this internally
```

Linii importante: 50-124

```
50 |   }, [])
51 |
52 |   if (!editor) return null
53 |
54 |   return (
55 |     <div ref={editorRef} style={{ position: 'relative' }}>
56 |       {/* Bubble formatting menu */}
57 |       <BubbleMenu
58 |         editor={editor}
59 |         tippyOptions={{ duration: 100 }}
60 |         className="bubble-menu"
61 |       >
62 |         <button
63 |           className={`bubble-menu-btn ${editor.isActive('bold') ? 'active' : ''}`}
64 |           onClick={() => editor.chain().focus().toggleBold().run()}
65 |           title="Bold (⌘B)"
66 |         >
67 |           <Bold size={14} />
68 |         </button>
69 |         <button
70 |           className={`bubble-menu-btn ${editor.isActive('italic') ? 'active' : ''}`}
71 |           onClick={() => editor.chain().focus().toggleItalic().run()}
72 |           title="Italic (⌘I)"
73 |         >
74 |           <Italic size={14} />
75 |         </button>
76 |         <button
77 |           className={`bubble-menu-btn ${editor.isActive('underline') ? 'active' : ''}`}
78 |           onClick={() => editor.chain().focus().toggleUnderline().run()}
79 |           title="Underline (⌘U)"
80 |         >
81 |           <UnderlineIcon size={14} />
82 |         </button>
83 |         <button
84 |           className={`bubble-menu-btn ${editor.isActive('strike') ? 'active' : ''}`}
85 |           onClick={() => editor.chain().focus().toggleStrike().run()}
86 |           title="Strikethrough"
87 |         >
88 |           <Strikethrough size={14} />
89 |         </button>
90 |         <div className="bubble-menu-divider" />
```

```
 91 |         <button
 92 |           className={`bubble-menu-btn ${editor.isActive('code') ? 'active' : ''}`}
 93 |           onClick={() => editor.chain().focus().toggleCode().run()}
 94 |           title="Inline code"
 95 |         >
 96 |           <Code size={14} />
 97 |         </button>
 98 |         <button
 99 |           className={`bubble-menu-btn ${editor.isActive('highlight') ? 'active' : ''}`}
100 |           onClick={() => editor.chain().focus().toggleHighlight().run()}
101 |           title="Highlight"
102 |         >
103 |           <Highlighter size={14} />
104 |         </button>
105 |         <div className="bubble-menu-divider" />
106 |         <button
107 |           className="bubble-menu-btn"
108 |           onClick={() => {
109 |             const url = window.prompt('URL:')
110 |             if (url) editor.chain().focus().setLink({ href: url }).run()
111 |           }}
112 |           title="Add link"
113 |         >
114 |           <LinkIcon size={14} />
115 |         </button>
116 |       </BubbleMenu>
117 |
118 |       <SlashMenu editor={editor} />
119 |
120 |       <EditorContent editor={editor} onKeyDown={handleKeyDown} />
121 |     </div>
122 |   )
123 | }
124 |
```

apps/web/src/components/SlashMenu.tsx

Command palette în editor. Filtrează comenzi după `/`, navighează cu tastatura și inserează tipuri de conținut. Este pregătirea UX pentru block editor.

Linii importante: 1-31

```
 1 | import { type Editor } from '@tiptap/react'
 2 | import { useState, useEffect, useRef, useCallback } from 'react'
 3 | import {
 4 |   Heading1, Heading2, Heading3, List, ListOrdered,
 5 |   CheckSquare, Quote, Code, Minus,
 6 |   Type, Pencil
 7 | } from 'lucide-react'
 8 |
 9 | interface SlashMenuProps {
10 |   editor: Editor
11 | }
12 |
13 | interface MenuItem {
14 |   label: string
15 |   hint: string
16 |   icon: React.ReactNode
17 |   group: string
18 |   command: () => void
19 | }
20 |
21 | export function SlashMenu({ editor }: SlashMenuProps) {
22 |   const [open, setOpen] = useState(false)
23 |   const [query, setQuery] = useState('')
24 |   const [activeIndex, setActiveIndex] = useState(0)
25 |   const [position, setPosition] = useState({ top: 0, left: 0 })
26 |   const menuRef = useRef<HTMLDivElement>(null)
27 |
28 |   const groups: { label: string; items: MenuItem[] }[] = [
29 |     {
30 |       label: 'Text',
31 |       items: [
```

Linii importante: 105-173

```
105 |   const filteredGroups = groups.map((g) => ({
106 |     ...g,
107 |     items: g.items.filter(
108 |       (item) =>
109 |         !query ||
110 |         item.label.toLowerCase().includes(query.toLowerCase()) ||
111 |         item.hint.toLowerCase().includes(query.toLowerCase())
112 |     ),
113 |   })).filter((g) => g.items.length > 0)
114 |
115 |   const filteredItems = filteredGroups.flatMap((g) => g.items)
```

```
116 |
117 |   const handleKeyDown = useCallback(
118 |     (e: KeyboardEvent) => {
119 |       if (!open) {
120 |         if (e.key === '/') {
121 |           // Get cursor position
122 |           const selection = window.getSelection()
123 |           if (selection && selection.rangeCount > 0) {
124 |             const range = selection.getRangeAt(0)
125 |             const rect = range.getBoundingClientRect()
126 |             setPosition({ top: rect.bottom + 8, left: rect.left })
127 |           }
128 |           setOpen(true)
129 |           setQuery('')
130 |           setActiveIndex(0)
131 |         }
132 |         return
133 |       }
134 |
135 |       if (e.key === 'Escape') { setOpen(false); return }
136 |       if (e.key === 'ArrowDown') {
137 |         e.preventDefault()
138 |         setActiveIndex((i) => (i + 1) % filteredItems.length)
139 |         return
140 |       }
141 |       if (e.key === 'ArrowUp') {
142 |         e.preventDefault()
143 |         setActiveIndex((i) => (i - 1 + filteredItems.length) % filteredItems.length)
144 |         return
145 |       }
146 |       if (e.key === 'Enter') {
147 |         e.preventDefault()
148 |         filteredItems[activeIndex]?.command()
149 |         setOpen(false)
150 |         // Delete the slash
151 |         editor.chain().focus().deleteRange({
152 |           from: editor.state.selection.from - query.length - 1,
153 |           to: editor.state.selection.from,
154 |         }).run()
155 |         return
156 |       }
157 |       if (e.key === 'Backspace') {
158 |         if (query === '') { setOpen(false); return }
159 |         setQuery((q) => q.slice(0, -1))
160 |         setActiveIndex(0)
161 |         return
162 |       }
163 |       if (e.key.length === 1) {
164 |         setQuery((q) => q + e.key)
165 |         setActiveIndex(0)
```

```
166 |       }
167 |     },
168 |     [open, query, filteredItems, activeIndex, editor]
169 |   )
170 |
171 |   useEffect(() => {
172 |     window.addEventListener('keydown', handleKeyDown, true)
173 |     return () => window.removeEventListener('keydown', handleKeyDown, true)
```

Linii importante: 192-237

```
192 |   return (
193 |     <div
194 |       ref={menuRef}
195 |       className="slash-menu"
196 |       style={{ position: 'fixed', top: position.top, left: position.left }}
197 |       role="listbox"
198 |     >
199 |       {filteredGroups.map((group) => (
200 |         <div key={group.label}>
201 |           <div className="slash-menu-group-label">{group.label}</div>
202 |           {group.items.map((item) => {
203 |             const idx = itemIndex++
204 |             return (
205 |               <div
206 |                 key={item.label}
207 |                 className={`slash-menu-item ${activeIndex === idx ? 'active' : ''}`}
208 |                 role="option"
209 |                 aria-selected={activeIndex === idx}
210 |                 onMouseEnter={() => setActiveIndex(idx)}
211 |                 onClick={() => {
212 |                   item.command()
213 |                   setOpen(false)
214 |                   editor.chain().focus().deleteRange({
215 |                     from: editor.state.selection.from - query.length - 1,
216 |                     to: editor.state.selection.from,
217 |                   }).run()
218 |                 }}
219 |               >
220 |                 <div className="slash-menu-item-icon">{item.icon}</div>
221 |                 <div className="slash-menu-item-text">
222 |                   <div className="label">{item.label}</div>
223 |                   <div className="hint">{item.hint}</div>
224 |                 </div>
225 |               </div>
226 |             )
227 |           })}
228 |         </div>
229 |       ))}
230 |       {query && (
```

```
231 |         <div style={{ padding: '6px 12px', fontSize: 11, color: 'var(--color-text-tertiary)', borderTop: '1px solid var(--color-border-subtle)' }}>
232 |           Searching: "<strong>{query}</strong>"
233 |         </div>
234 |       )}
235 |     </div>
236 |   )
237 | }
```

apps/web/src/components/CanvasBlock.tsx

Canvas de desen. Gestionează pointer events, stroke-uri cu presiune, randare pe HTML Canvas, unelte pen/highlighter/eraser/lasso și undo/redo local. `perfect-freehand` generează conturul stroke-ului; sincronizarea Yjs nu este implementată încă.

Linii importante: 1-36

```
 1 | import { useState, useRef, useEffect, useCallback } from 'react'
 2 | import {
 3 |   MousePointer2, PenLine, Highlighter, Eraser,
 4 |   Lasso, Shapes, Type, Undo2, Redo2, GripVertical
 5 | } from 'lucide-react'
 6 | import { getStroke } from 'perfect-freehand'
 7 |
 8 | type Tool = 'select' | 'pen' | 'highlighter' | 'eraser' | 'lasso' | 'shapes' | 'text'
 9 |
10 | interface Stroke {
11 |   id: string
12 |   tool: 'pen' | 'highlighter'
13 |   color: string
14 |   width: number
15 |   opacity: number
16 |   points: [number, number, number][]
17 | }
18 |
19 | const INK_COLORS = [
20 |   '#1a1a1a', '#6941C6', '#175CD3', '#067647', '#B54708', '#B42318',
21 |   '#1D2939', '#7F56D9', '#2E90FA', '#12B76A', '#F79009', '#F04438',
22 | ]
23 |
24 | export function CanvasBlock() {
25 |   const canvasRef = useRef<HTMLCanvasElement>(null)
26 |   const [activeTool, setActiveTool] = useState<Tool>('pen')
27 |   const [activeColor, setActiveColor] = useState('#1a1a1a')
28 |   const [strokeWidth, setStrokeWidth] = useState(4)
29 |   const [strokes, setStrokes] = useState<Stroke[]>([])
30 |   const [liveStroke, setLiveStroke] = useState<[number, number, number][]>([])
31 |   const [isDrawing, setIsDrawing] = useState(false)
32 |   const [history, setHistory] = useState<Stroke[][]>([[]])
33 |   const [historyIndex, setHistoryIndex] = useState(0)
34 |
35 |   const WIDTH = 800
36 |   const HEIGHT = 440
```

Linii importante: 55-170

```
55 |   const render = useCallback(() => {
56 |     const canvas = canvasRef.current
57 |     if (!canvas) return
58 |     const ctx = canvas.getContext('2d')
59 |     if (!ctx) return
```

```
60 |
61 |     ctx.clearRect(0, 0, WIDTH, HEIGHT)
62 |
63 |     // Draw grid
64 |     ctx.strokeStyle = 'rgba(0,0,0,0.04)'
65 |     ctx.lineWidth = 1
66 |     for (let x = 0; x < WIDTH; x += 20) {
67 |       ctx.beginPath(); ctx.moveTo(x, 0); ctx.lineTo(x, HEIGHT); ctx.stroke()
68 |     }
69 |     for (let y = 0; y < HEIGHT; y += 20) {
70 |       ctx.beginPath(); ctx.moveTo(0, y); ctx.lineTo(WIDTH, y); ctx.stroke()
71 |     }
72 |
73 |     // Draw s-plane axes and stable region if enabled
74 |     if (showAxes) {
75 |       const originX = WIDTH / 2 + 60
76 |       const originY = HEIGHT / 2
77 |
78 |       // LHP Stable region tint
79 |       ctx.fillStyle = 'rgba(18, 183, 106, 0.04)'
80 |       ctx.fillRect(0, 0, originX, HEIGHT)
81 |
82 |       // Stable region label
83 |       ctx.fillStyle = 'rgba(18, 183, 106, 0.7)'
84 |       ctx.font = '500 11px Inter, sans-serif'
85 |       ctx.fillText('Stable Region: Re(s) < 0', 24, 28)
86 |
87 |       // Axes lines
88 |       ctx.strokeStyle = 'rgba(102, 112, 133, 0.4)'
89 |       ctx.lineWidth = 1.5
90 |
91 |       // Real axis
92 |       ctx.beginPath()
93 |       ctx.moveTo(30, originY)
94 |       ctx.lineTo(WIDTH - 30, originY)
95 |       ctx.stroke()
96 |
97 |       // Imaginary axis
98 |       ctx.beginPath()
99 |       ctx.moveTo(originX, 20)
100 |      ctx.lineTo(originX, HEIGHT - 20)
101 |      ctx.stroke()
102 |
103 |      // Axis labels
104 |      ctx.fillStyle = 'var(--color-text-secondary)'
105 |      ctx.font = '600 12px Inter, sans-serif'
106 |      ctx.fillText('σ (Real)', WIDTH - 75, originY - 8)
107 |      ctx.fillText('jω (Imaginary)', originX + 8, 30)
108 |
109 |      // Stable poles markers (X)
```

```
110 |       const poles = [
111 |         { x: originX - 120, y: originY - 70 },
112 |         { x: originX - 120, y: originY + 70 },
113 |         { x: originX - 220, y: originY },
114 |       ]
115 |       ctx.strokeStyle = '#B42318'
116 |       ctx.lineWidth = 2
117 |       poles.forEach((p) => {
118 |         ctx.beginPath()
119 |         ctx.moveTo(p.x - 6, p.y - 6); ctx.lineTo(p.x + 6, p.y + 6)
120 |         ctx.moveTo(p.x + 6, p.y - 6); ctx.lineTo(p.x - 6, p.y + 6)
121 |         ctx.stroke()
122 |       })
123 |     }
124 |
125 |     // Draw committed strokes
126 |     strokes.forEach((stroke) => {
127 |       drawStroke(ctx, stroke.points, stroke.color, stroke.width, stroke.opacity, stroke.tool ===
'highlighter')
128 |     })
129 |
130 |     // Draw live stroke
131 |     if (liveStroke.length > 1) {
132 |       drawStroke(ctx, liveStroke, activeColor, strokeWidth, activeTool === 'highlighter' ? 0.35 : 1,
activeTool === 'highlighter')
133 |     }
134 |   }, [strokes, liveStroke, activeColor, strokeWidth, activeTool, showAxes])
135 |
136 |   useEffect(() => { render() }, [render])
137 |
138 |   function drawStroke(
139 |     ctx: CanvasRenderingContext2D,
140 |     points: [number, number, number][],
141 |     color: string,
142 |     width: number,
143 |     opacity: number,
144 |     isHighlighter: boolean
145 |   ) {
146 |     if (points.length < 2) return
147 |
148 |     const outlinePoints = getStroke(points, {
149 |       size: isHighlighter ? width * 4 : width * 2,
150 |       thinning: isHighlighter ? 0 : 0.5,
151 |       smoothing: 0.5,
152 |       streamline: 0.5,
153 |       simulatePressure: true,
154 |     })
155 |
156 |     if (outlinePoints.length < 2) return
157 |
```

```
158 |     ctx.save()
159 |     ctx.globalAlpha = opacity
160 |     ctx.fillStyle = color
161 |
162 |     const path = new Path2D()
163 |     path.moveTo(outlinePoints[0][0], outlinePoints[0][1])
164 |     for (let i = 1; i < outlinePoints.length; i++) {
165 |       path.lineTo(outlinePoints[i][0], outlinePoints[i][1])
166 |     }
167 |     path.closePath()
168 |     ctx.fill(path)
169 |     ctx.restore()
170 |   }
```

Linii importante: 173-242

```
173 |     const canvas = canvasRef.current!
174 |     const rect = canvas.getBoundingClientRect()
175 |     const scaleX = WIDTH / rect.width
176 |     const scaleY = HEIGHT / rect.height
177 |     return [
178 |       (e.clientX - rect.left) * scaleX,
179 |       (e.clientY - rect.top) * scaleY,
180 |       e.pressure || 0.5,
181 |     ]
182 |   }
183 |
184 |   function onPointerDown(e: React.PointerEvent) {
185 |     if (activeTool === 'select') return
186 |     e.currentTarget.setPointerCapture(e.pointerId)
187 |     setIsDrawing(true)
188 |     setLiveStroke([getCanvasPoint(e)])
189 |   }
190 |
191 |   function onPointerMove(e: React.PointerEvent) {
192 |     if (!isDrawing) return
193 |     setLiveStroke((pts) => [...pts, getCanvasPoint(e)])
194 |   }
195 |
196 |   function onPointerUp() {
197 |     if (!isDrawing || activeTool === 'select') return
198 |     setIsDrawing(false)
199 |
200 |     if (liveStroke.length > 1 && activeTool !== 'eraser') {
201 |       const newStroke: Stroke = {
202 |         id: crypto.randomUUID(),
203 |         tool: activeTool === 'highlighter' ? 'highlighter' : 'pen',
204 |         color: activeColor,
205 |         width: strokeWidth,
206 |         opacity: activeTool === 'highlighter' ? 0.35 : 1,
```

```
207 |         points: liveStroke,
208 |       }
209 |       const newStrokes = [...strokes, newStroke]
210 |       setStrokes(newStrokes)
211 |
212 |       // Update history
213 |       const newHistory = history.slice(0, historyIndex + 1)
214 |       newHistory.push(newStrokes)
215 |       setHistory(newHistory)
216 |       setHistoryIndex(newHistory.length - 1)
217 |     }
218 |
219 |     if (activeTool === 'eraser' && liveStroke.length > 0) {
220 |       // Erase strokes that are near the eraser path
221 |       const [ex, ey] = liveStroke[liveStroke.length - 1]
222 |       setStrokes((prev) => prev.filter((stroke) => {
223 |         return !stroke.points.some(([x, y]) => Math.hypot(x - ex, y - ey) < 20)
224 |       }))
225 |     }
226 |
227 |     setLiveStroke([])
228 |   }
229 |
230 |   const undo = () => {
231 |     if (historyIndex <= 0) return
232 |     const idx = historyIndex - 1
233 |     setHistoryIndex(idx)
234 |     setStrokes(history[idx])
235 |   }
236 |
237 |   const redo = () => {
238 |     if (historyIndex >= history.length - 1) return
239 |     const idx = historyIndex + 1
240 |     setHistoryIndex(idx)
241 |     setStrokes(history[idx])
242 |   }
```

Linii importante: 244-435

```
244 |   const tools: { id: Tool; icon: React.ReactNode; label: string; shortcut: string }[] = [
245 |     { id: 'select', icon: <MousePointer2 size={15} />, label: 'Select', shortcut: 'S' },
246 |     { id: 'pen', icon: <PenLine size={15} />, label: 'Pen', shortcut: 'P' },
247 |     { id: 'highlighter', icon: <Highlighter size={15} />, label: 'Highlighter', shortcut: 'H' },
248 |     { id: 'eraser', icon: <Eraser size={15} />, label: 'Eraser', shortcut: 'E' },
249 |     { id: 'lasso', icon: <Lasso size={15} />, label: 'Lasso', shortcut: 'L' },
250 |     { id: 'shapes', icon: <Shapes size={15} />, label: 'Shapes', shortcut: '' },
251 |     { id: 'text', icon: <Type size={15} />, label: 'Text', shortcut: 'T' },
252 |   ]
253 |
254 |   // Keyboard shortcuts
```

```
 255 |   useEffect(() => {
 256 |     const handler = (e: KeyboardEvent) => {
 257 |       if (e.target instanceof HTMLInputElement) return
 258 |       const map: Record<string, Tool> = { p: 'pen', h: 'highlighter', e: 'eraser', l: 'lasso', s: 'select',
t: 'text' }
 259 |       if (map[e.key.toLowerCase()]) setActiveTool(map[e.key.toLowerCase()])
 260 |       if ((e.metaKey || e.ctrlKey) && e.key === 'z') { e.shiftKey ? redo() : undo() }
 261 |     }
 262 |     window.addEventListener('keydown', handler)
 263 |     return () => window.removeEventListener('keydown', handler)
 264 |   }, [undo, redo])
 265 |
 266 |   return (
 267 |     <div className="canvas-block">
 268 |       {/* Canvas */}
 269 |       <canvas
 270 |         ref={canvasRef}
 271 |         width={WIDTH}
 272 |         height={HEIGHT}
 273 |         style={{
 274 |           width: '100%',
 275 |           height: 'auto',
 276 |           cursor: activeTool === 'select' ? 'default'
 277 |             : activeTool === 'eraser' ? 'cell'
 278 |             : 'crosshair',
 279 |           display: 'block',
 280 |           maxHeight: 440,
 281 |         }}
 282 |         onPointerDown={onPointerDown}
 283 |         onPointerMove={onPointerMove}
 284 |         onPointerUp={onPointerUp}
 285 |       />
 286 |
 287 |       {/* Drawing Palette (bottom) */}
 288 |       <div
 289 |         className="canvas-toolbar"
 290 |         style={{
 291 |           display: 'flex',
 292 |           alignItems: 'center',
 293 |           justifyContent: 'space-between',
 294 |           padding: '6px 12px',
 295 |           background: 'var(--color-bg-page)',
 296 |           borderTop: '1px solid var(--color-border-subtle)',
 297 |           borderRadius: '0 0 12px 12px',
 298 |         }}
 299 |       >
 300 |         {/* Left tools */}
 301 |         <div style={{ display: 'flex', alignItems: 'center', gap: 4 }}>
 302 |           <GripVertical size={14} style={{ color: 'var(--color-text-disabled)', cursor: 'grab', marginRight:
2 }} />
```

```
303 |           {tools.map((tool) => (
304 |             <button
305 |               key={tool.id}
306 |               className={`canvas-tool-btn ${activeTool === tool.id ? 'active' : ''}`}
307 |               onClick={() => setActiveTool(tool.id)}
308 |               data-tooltip={`${tool.label}${tool.shortcut ? ` (${tool.shortcut})` : ''}`}
309 |               aria-label={tool.label}
310 |             >
311 |               {tool.icon}
312 |             </button>
313 |           ))}
314 |         </div>
315 |
316 |         {/* Center colors & width */}
317 |         <div style={{ display: 'flex', alignItems: 'center', gap: 6 }}>
318 |           {INK_COLORS.slice(0, 6).map((color) => (
319 |             <button
320 |               key={color}
321 |               className={`ink-swatch ${activeColor === color ? 'selected' : ''}`}
322 |               style={{ background: color }}
323 |               onClick={() => setActiveColor(color)}
324 |               aria-label={`Color ${color}`}
325 |             />
326 |           ))}
327 |
328 |           <select
329 |             value={strokeWidth}
330 |             onChange={(e) => setStrokeWidth(Number(e.target.value))}
331 |             style={{
332 |               border: '1px solid var(--color-border-default)',
333 |               borderRadius: 6,
334 |               padding: '2px 4px',
335 |               fontSize: 11,
336 |               background: 'var(--color-bg-page)',
337 |               color: 'var(--color-text-primary)',
338 |               cursor: 'pointer',
339 |               outline: 'none',
340 |               marginLeft: 4,
341 |             }}
342 |             aria-label="Stroke width"
343 |           >
344 |             <option value={2}>2 px</option>
345 |             <option value={4}>4 px</option>
346 |             <option value={6}>6 px</option>
347 |             <option value={10}>10 px</option>
348 |           </select>
349 |         </div>
350 |
351 |         {/* Right actions */}
352 |         <div style={{ display: 'flex', alignItems: 'center', gap: 4 }}>
```

```
353 |           <button
354 |             className="canvas-tool-btn"
355 |             onClick={undo}
356 |             disabled={historyIndex <= 0}
357 |             data-tooltip="Undo (⌘Z)"
358 |             aria-label="Undo"
359 |           >
360 |             <Undo2 size={14} />
361 |           </button>
362 |           <button
363 |             className="canvas-tool-btn"
364 |             onClick={redo}
365 |             disabled={historyIndex >= history.length - 1}
366 |             data-tooltip="Redo (⌘⇧Z)"
367 |             aria-label="Redo"
368 |           >
369 |             <Redo2 size={14} />
370 |           </button>
371 |
372 |           <div style={{ width: 1, height: 18, background: 'var(--color-border-subtle)', margin: '0 4px' }} />
373 |
374 |           <button
375 |             className={`btn btn-xs ${showAxes ? 'btn-primary' : 'btn-ghost'}`}
376 |             onClick={() => setShowAxes(!showAxes)}
377 |             data-tooltip="Toggle s-plane coordinate axes"
378 |             style={{ fontSize: 11, padding: '3px 7px' }}
379 |           >
380 |             s-plane
381 |           </button>
382 |
383 |           <button
384 |             className="btn btn-xs btn-secondary"
385 |             onClick={() => {
386 |               const originX = WIDTH / 2 + 60
387 |               const originY = HEIGHT / 2
388 |               const branch1: Stroke = {
389 |                 id: `rl-1-${Date.now()}`,
390 |                 tool: 'pen',
391 |                 color: '#7F56D9',
392 |                 width: 3,
393 |                 opacity: 0.9,
394 |                 points: [
395 |                   [originX - 120, originY - 70, 0.6],
396 |                   [originX - 90, originY - 40, 0.7],
397 |                   [originX - 50, originY, 0.8],
398 |                   [originX, originY - 80, 0.7],
399 |                   [originX + 30, originY - 140, 0.6],
400 |                 ],
401 |               }
402 |               const branch2: Stroke = {
```

```
                id: `rl-2-${Date.now()}`,
                tool: 'pen',
                color: '#7F56D9',
                width: 3,
                opacity: 0.9,
                points: [
                  [originX - 120, originY + 70, 0.6],
                  [originX - 90, originY + 40, 0.7],
                  [originX - 50, originY, 0.8],
                  [originX, originY + 80, 0.7],
                  [originX + 30, originY + 140, 0.6],
                ],
              }
              setStrokes((prev) => [...prev, branch1, branch2])
            }}
            data-tooltip="Plot Root Locus trajectories"
            style={{ fontSize: 11, padding: '3px 7px' }}
          >
            Plot Locus
          </button>

          <button
            className="btn btn-xs btn-ghost"
            onClick={() => setStrokes([])}
            style={{ fontSize: 11, color: 'var(--color-text-tertiary)', padding: '3px 6px' }}
          >
            Clear
          </button>
        </div>
      </div>
    </div>
  )
}
```

apps/web/src/components/CopilotPanel.tsx

Interfață Copilot AI simulată. Are tab-uri Ask/Drawing/Tasks/Summary, mesaje demo, recunoaștere simulată și task-uri locale. Nu există `fetch`, endpoint `/ai/ask`, embeddings sau cheie AI în client.

Linii importante: 1-38

```
  1 | import { useState, useRef, useEffect } from 'react'
  2 | import { Sparkles, X, Send, ChevronRight, Loader, Pencil, FileText, Check } from 'lucide-react'
  3 | import { useAppStore } from '../store/appStore'
  4 |
  5 | type CopilotTab = 'ask' | 'drawing' | 'tasks' | 'summary'
  6 |
  7 | interface Message {
  8 |   id: string
  9 |   role: 'user' | 'ai'
 10 |   content: string
 11 |   citations?: { index: number; title: string; quote: string }[]
 12 | }
 13 |
 14 | const DEMO_MESSAGES: Message[] = [
 15 |   {
 16 |     id: '1', role: 'user',
 17 |     content: 'Explain feedback stability from my notes',
 18 |   },
 19 |   {
 20 |     id: '2', role: 'ai',
 21 |     content: 'Based on your notes on **Feedback Stability** [1], a closed-loop system is stable when all
poles of the transfer function lie in the left half of the complex s-plane.\n\nThe key criterion comes from your
Bode Plots notes [2]: gain margin and phase margin must both be positive for stability.',
 22 |     citations: [
 23 |       { index: 1, title: 'Feedback Stability', quote: 'all poles must lie in the left half of the complex s-
plane' },
 24 |       { index: 2, title: 'Bode Plots', quote: 'gain margin and phase margin must both be positive' },
 25 |     ],
 26 |   },
 27 | ]
 28 |
 29 | export function CopilotPanel() {
 30 |   const { toggleCopilot } = useAppStore()
 31 |   const [activeTab, setActiveTab] = useState<CopilotTab>('ask')
 32 |   const [messages, setMessages] = useState<Message[]>(DEMO_MESSAGES)
 33 |   const [input, setInput] = useState('')
 34 |   const [isLoading, setIsLoading] = useState(false)
 35 |   const [segmentedMode, setSegmentedMode] = useState<'insert' | 'replace'>('insert')
 36 |   const [recognitionLoading, setRecognitionLoading] = useState(false)
 37 |   const [recognizedResult, setRecognizedResult] = useState<string | null>(null)
 38 |   const [tasksDone, setTasksDone] = useState<Record<string, boolean>>({
```

Linii importante: 45-112

```
  45 |
  46 |   useEffect(() => {
  47 |     messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' })
  48 |   }, [messages])
  49 |
  50 |   async function handleSend() {
  51 |     if (!input.trim() || isLoading) return
  52 |
  53 |     const userMsg: Message = { id: crypto.randomUUID(), role: 'user', content: input }
  54 |     setMessages((m) => [...m, userMsg])
  55 |     setInput('')
  56 |     setIsLoading(true)
  57 |
  58 |     // Simulate AI response
  59 |     await new Promise((r) => setTimeout(r, 1200))
  60 |
  61 |     const aiMsg: Message = {
  62 |       id: crypto.randomUUID(),
  63 |       role: 'ai',
  64 |       content: 'Based on your notes, this topic is covered in the context of control systems theory. The
relationship between input and output stability is defined by the system\'s transfer function poles [1].\n\nI found
3 relevant sections in your notes that discuss this.',
  65 |       citations: [
  66 |         { index: 1, title: 'Feedback Stability', quote: 'poles of the closed-loop transfer function' },
  67 |       ],
  68 |     }
  69 |     setMessages((m) => [...m, aiMsg])
  70 |     setIsLoading(false)
  71 |   }
  72 |
  73 |   function handleKeyDown(e: React.KeyboardEvent<HTMLTextAreaElement>) {
  74 |     if (e.key === 'Enter' && !e.shiftKey) {
  75 |       e.preventDefault()
  76 |       handleSend()
  77 |     }
  78 |   }
  79 |
  80 |   // Auto-resize textarea
  81 |   function handleInput(e: React.ChangeEvent<HTMLTextAreaElement>) {
  82 |     setInput(e.target.value)
  83 |     const el = e.target
  84 |     el.style.height = 'auto'
  85 |     el.style.height = Math.min(el.scrollHeight, 120) + 'px'
  86 |   }
  87 |
  88 |   function renderMessage(msg: Message) {
  89 |     if (msg.role === 'user') {
  90 |       return (
  91 |         <div key={msg.id} className="copilot-message user">
```

```
 92 |           {msg.content}
 93 |         </div>
 94 |       )
 95 |     }
 96 |
 97 |     // Parse bold markdown
 98 |     const parts = msg.content.split(/(\*\*[^*]+\*\*|\[\d+\])/g)
 99 |     const rendered = parts.map((part, i) => {
100 |       if (part.startsWith('**') && part.endsWith('**')) {
101 |         return <strong key={i}>{part.slice(2, -2)}</strong>
102 |       }
103 |       const citMatch = part.match(/^\[(\d+)\]$/)
104 |       if (citMatch) {
105 |         const idx = parseInt(citMatch[1])
106 |         const cit = msg.citations?.find((c) => c.index === idx)
107 |         return (
108 |           <button
109 |             key={i}
110 |             className="copilot-citation"
111 |             onClick={() => {}}
112 |             title={cit?.quote}
```

Linii importante: 163-238

```
163 |   return (
164 |     <aside className="copilot-panel">
165 |       {/* Header */}
166 |       <div className="copilot-header">
167 |         <div className="copilot-header-title">
168 |           <Sparkles size={15} className="copilot-sparkle" />
169 |           AI Co-pilot
170 |         </div>
171 |         <button
172 |           className="btn btn-icon btn-ghost"
173 |           onClick={toggleCopilot}
174 |           aria-label="Close co-pilot"
175 |           style={{ width: 28, height: 28 }}
176 |         >
177 |           <X size={15} />
178 |         </button>
179 |       </div>
180 |
181 |       {/* Tabs */}
182 |       <div className="copilot-tabs" style={{ display: 'grid', gridTemplateColumns: 'repeat(4, 1fr)', padding:
'0 8px' }}>
183 |         {(['ask', 'drawing', 'tasks', 'summary'] as CopilotTab[]).map((tab) => (
184 |           <button
185 |             key={tab}
186 |             className={`copilot-tab ${activeTab === tab ? 'active' : ''}`}
187 |             onClick={() => setActiveTab(tab)}
```

```
188 |             style={{ padding: '8px 4px', fontSize: 12, textAlign: 'center' }}
189 |           >
190 |             {tab === 'ask' ? 'Ask' : tab === 'drawing' ? 'Drawing' : tab === 'tasks' ? 'Tasks' : 'Summary'}
191 |           </button>
192 |         ))}
193 |       </div>
194 |
195 |       {/* Body */}
196 |       {activeTab === 'ask' ? (
197 |         <>
198 |           <div className="copilot-body">
199 |             {messages.length === 0 ? (
200 |               <div className="copilot-empty-state">
201 |                 <div className="copilot-empty-icon">✧</div>
202 |                 <div>
203 |                   <p style={{ fontWeight: 600, color: 'var(--color-text-primary)', marginBottom: 4 }}>
204 |                     Ask about your notes
205 |                   </p>
206 |                   <p style={{ fontSize: 13, color: 'var(--color-text-secondary)', lineHeight: 1.5 }}>
207 |                     I'll answer only from what's in your notebooks, with citations.
208 |                   </p>
209 |                 </div>
210 |               </div>
211 |             ) : (
212 |               messages.map(renderMessage)
213 |             )}
214 |             {isLoading && (
215 |               <div className="copilot-message ai" style={{ display: 'flex', alignItems: 'center', gap: 8 }}>
216 |                 <Loader size={14} className="animate-spin" style={{ color: 'var(--color-accent)' }} />
217 |                 <span style={{ color: 'var(--color-text-tertiary)', fontSize: 13 }}>Searching your notes…
</span>
218 |               </div>
219 |             )}
220 |             <div ref={messagesEndRef} />
221 |           </div>
222 |
223 |           {/* Input */}
224 |           <div className="copilot-input-area">
225 |             <div className="copilot-input-wrapper">
226 |               <textarea
227 |                 ref={textareaRef}
228 |                 className="copilot-input"
229 |                 placeholder="Ask your notes anything…"
230 |                 value={input}
231 |                 onChange={handleInput}
232 |                 onKeyDown={handleKeyDown}
233 |                 rows={1}
234 |                 style={{ resize: 'none' }}
235 |               />
236 |               <button
```

```
237 |                 className="btn btn-primary btn-sm"
238 |                 onClick={handleSend}
```

Linii importante: 280-526

```
280 |               style={{ justifyContent: 'flex-start', padding: '10px 14px' }}
281 |               onClick={() => {
282 |                 setRecognitionLoading(true)
283 |                 setTimeout(() => {
284 |                   setRecognizedResult('1 + G(s)H(s) = 0\nClosed-loop characteristic equation: s² + 2ζωₙs +
ωₙ² = 0')
285 |                   setRecognitionLoading(false)
286 |                 }, 1000)
287 |               }}
288 |             >
289 |               <FileText size={15} style={{ color: 'var(--color-brand-600)' }} />
290 |               <div style={{ textAlign: 'left' }}>
291 |                 <div style={{ fontSize: 13, fontWeight: 600 }}>Convert handwriting to text</div>
292 |                 <div style={{ fontSize: 11, color: 'var(--color-text-tertiary)' }}>Transcribe formulas &amp;
annotations</div>
293 |               </div>
294 |             </button>
295 |
296 |             <button
297 |               className="btn btn-secondary"
298 |               style={{ justifyContent: 'flex-start', padding: '10px 14px' }}
299 |               onClick={() => {
300 |                 setRecognitionLoading(true)
301 |                 setTimeout(() => {
302 |                   setRecognizedResult('Root Locus plot: 2 open-loop poles at s = -1 ± 2j and 1 zero at s =
-3. Trajectory crosses imaginary axis at K_crit = 4.2.')
303 |                   setRecognitionLoading(false)
304 |                 }, 1000)
305 |               }}
306 |             >
307 |               <Sparkles size={15} style={{ color: 'var(--color-brand-600)' }} />
308 |               <div style={{ textAlign: 'left' }}>
309 |                 <div style={{ fontSize: 13, fontWeight: 600 }}>Describe this sketch</div>
310 |                 <div style={{ fontSize: 11, color: 'var(--color-text-tertiary)' }}>Generate technical
explanation</div>
311 |               </div>
312 |             </button>
313 |           </div>
314 |
315 |           {/* Loading state */}
316 |           {recognitionLoading && (
317 |             <div
318 |               style={{
319 |                 padding: 16,
320 |                 borderRadius: 10,
```

```
                background: 'var(--color-brand-50)',
                display: 'flex',
                alignItems: 'center',
                gap: 10,
              }}
            >
              <Loader size={16} className="animate-spin" style={{ color: 'var(--color-brand-600)' }} />
              <span style={{ fontSize: 13, color: 'var(--color-brand-800)' }}>
                Analyzing sketch with vision model…
              </span>
            </div>
          )}

          {/* Recognition result card */}
          {recognizedResult && (
            <div
              style={{
                padding: 14,
                borderRadius: 10,
                border: '1px solid var(--color-border-default)',
                background: 'var(--color-bg-page)',
                display: 'flex',
                flexDirection: 'column',
                gap: 12,
              }}
            >
              <div className="flex items-center justify-between">
                <span style={{ fontSize: 12, fontWeight: 600, color: 'var(--color-text-primary)' }}>
                  Recognized Content
                </span>
                <span className="badge badge-success" style={{ fontSize: 10 }}>
                  Confidence: 96%
                </span>
              </div>

              <pre
                style={{
                  background: 'var(--color-bg-subtle)',
                  padding: 10,
                  borderRadius: 6,
                  fontSize: 12,
                  color: 'var(--color-text-primary)',
                  whiteSpace: 'pre-wrap',
                  margin: 0,
                  border: '1px solid var(--color-border-subtle)',
                }}
              >
                {recognizedResult}
              </pre>

```

```
 371 |               {/* Segmented Control: Insert vs Replace */}
 372 |               <div
 373 |                 style={{
 374 |                   display: 'flex',
 375 |                   background: 'var(--color-bg-subtle)',
 376 |                   borderRadius: 8,
 377 |                   padding: 3,
 378 |                   border: '1px solid var(--color-border-subtle)',
 379 |                 }}
 380 |               >
 381 |                 <button
 382 |                   style={{
 383 |                     flex: 1,
 384 |                     border: 'none',
 385 |                     background: segmentedMode === 'insert' ? 'var(--color-bg-page)' : 'transparent',
 386 |                     boxShadow: segmentedMode === 'insert' ? '0 1px 3px rgba(0,0,0,0.06)' : 'none',
 387 |                     borderRadius: 6,
 388 |                     padding: '6px 0',
 389 |                     fontSize: 12,
 390 |                     fontWeight: 600,
 391 |                     color: segmentedMode === 'insert' ? 'var(--color-brand-700)' : 'var(--color-text-
tertiary)',
 392 |                     cursor: 'pointer',
 393 |                   }}
 394 |                   onClick={() => setSegmentedMode('insert')}
 395 |                 >
 396 |                   Insert as text
 397 |                 </button>
 398 |                 <button
 399 |                   style={{
 400 |                     flex: 1,
 401 |                     border: 'none',
 402 |                     background: segmentedMode === 'replace' ? 'var(--color-bg-page)' : 'transparent',
 403 |                     boxShadow: segmentedMode === 'replace' ? '0 1px 3px rgba(0,0,0,0.06)' : 'none',
 404 |                     borderRadius: 6,
 405 |                     padding: '6px 0',
 406 |                     fontSize: 12,
 407 |                     fontWeight: 600,
 408 |                     color: segmentedMode === 'replace' ? 'var(--color-brand-700)' : 'var(--color-text-
tertiary)',
 409 |                     cursor: 'pointer',
 410 |                   }}
 411 |                   onClick={() => setSegmentedMode('replace')}
 412 |                 >
 413 |                   Replace strokes
 414 |                 </button>
 415 |               </div>
 416 |
 417 |               {/* Apply button */}
 418 |               <button
```

```
419 |                 className="btn btn-primary btn-sm"
420 |                 style={{ justifyContent: 'center' }}
421 |                 onClick={() => {
422 |                   alert('Content successfully inserted into note!')
423 |                   setRecognizedResult(null)
424 |                 }}
425 |               >
426 |                 <Check size={13} />
427 |                 {segmentedMode === 'insert' ? 'Insert reviewed text into page' : 'Replace selected strokes'}
428 |               </button>
429 |             </div>
430 |           )}
431 |         </div>
432 |       ) : activeTab === 'tasks' ? (
433 |         <div className="copilot-body" style={{ display: 'flex', flexDirection: 'column', gap: 14 }}>
434 |           <div className="flex items-center justify-between" style={{ padding: '0 4px' }}>
435 |             <span style={{ fontSize: 13, fontWeight: 600, color: 'var(--color-text-primary)' }}>
436 |               Action Items in this Note
437 |             </span>
438 |             <span className="badge badge-brand" style={{ fontSize: 10 }}>
439 |               3 detected
440 |             </span>
441 |           </div>
442 |
443 |           <div style={{ display: 'flex', flexDirection: 'column', gap: 8 }}>
444 |             {[
445 |               { id: 't1', text: 'Review Routh-Hurwitz conditions and polynomial coefficients' },
446 |               { id: 't2', text: 'Simulate closed-loop root locus in MATLAB' },
447 |               { id: 't3', text: 'Write up analytical stability analysis section' },
448 |             ].map((task) => {
449 |               const done = tasksDone[task.id]
450 |               return (
451 |                 <div
452 |                   key={task.id}
453 |                   onClick={() => setTasksDone((prev) => ({ ...prev, [task.id]: !prev[task.id] }))}
454 |                   style={{
455 |                     padding: '10px 12px',
456 |                     borderRadius: 8,
457 |                     background: done ? 'var(--color-bg-subtle)' : 'var(--color-bg-page)',
458 |                     border: '1px solid var(--color-border-subtle)',
459 |                     display: 'flex',
460 |                     alignItems: 'flex-start',
461 |                     gap: 10,
462 |                     cursor: 'pointer',
463 |                   }}
464 |                 >
465 |                   <div
466 |                     style={{
467 |                       width: 16,
468 |                       height: 16,
```

```
                      borderRadius: 4,
                      border: done ? 'none' : '2px solid var(--color-border-strong)',
                      background: done ? 'var(--color-brand-600)' : 'transparent',
                      color: 'white',
                      display: 'flex',
                      alignItems: 'center',
                      justifyContent: 'center',
                      marginTop: 2,
                      flexShrink: 0,
                    }}
                  >
                    {done && <Check size={11} strokeWidth={3} />}
                  </div>
                  <span
                    style={{
                      fontSize: 12,
                      color: done ? 'var(--color-text-tertiary)' : 'var(--color-text-primary)',
                      textDecoration: done ? 'line-through' : 'none',
                      lineHeight: 1.4,
                    }}
                  >
                    {task.text}
                  </span>
                </div>
              )
            })}
          </div>
        </div>
      ) : (
        <div className="copilot-body">
          <div style={{ padding: '8px 4px', display: 'flex', flexDirection: 'column', gap: 14 }}>
            <p style={{ fontSize: 13, fontWeight: 600, color: 'var(--color-text-primary)', margin: 0 }}>
              Page Executive Summary
            </p>
            <div
              style={{
                fontSize: 13,
                color: 'var(--color-text-secondary)',
                lineHeight: 1.6,
                background: 'var(--color-bg-subtle)',
                padding: 14,
                borderRadius: 10,
                border: '1px solid var(--color-border-subtle)',
              }}
            >
              This note covers <strong>Feedback Stability</strong> in continuous-time control systems. Key
takeaways:
              <ul style={{ margin: '8px 0 0 16px', padding: 0 }}>
                <li>All closed-loop poles must reside strictly in the Left Half-Plane (LHP).</li>
                <li>Negative feedback alone does not assure stability without sufficient gain and phase
```

```
margins.</li>
 518 |                 <li>Root locus trajectories trace pole migration under varying loop gain K.</li>
 519 |               </ul>
 520 |             </div>
 521 |           </div>
 522 |         </div>
 523 |       )}
 524 |     </aside>
 525 |   )
 526 | }
```

apps/web/src/components/ProximitySyncDialog.tsx

Interfață de proximity sync. Prezintă peers și cod de pairing, dar datele și sincronizarea sunt mock. Documentația avertizează corect că WebRTC are nevoie de signaling.

Linii importante: 1-67

```
 1 | import { useState } from 'react'
 2 | import { Radio, X, Shield, QrCode, Smartphone, Laptop, Check, RefreshCw, Copy } from 'lucide-react'
 3 |
 4 | interface Peer {
 5 |   id: string
 6 |   name: string
 7 |   device: string
 8 |   type: 'tablet' | 'laptop' | 'phone'
 9 |   signal: 'strong' | 'good' | 'weak'
10 |   status: 'synced' | 'syncing' | 'available'
11 |   pages: number
12 |   ip: string
13 | }
14 |
15 | const INITIAL_PEERS: Peer[] = [
16 |   {
17 |     id: 'peer-1',
18 |     name: 'Maya Chen',
19 |     device: 'iPad Pro 11"',
20 |     type: 'tablet',
21 |     signal: 'strong',
22 |     status: 'synced',
23 |     pages: 3,
24 |     ip: '192.168.1.42:6190',
25 |   },
26 |   {
27 |     id: 'peer-2',
28 |     name: 'Radu Popescu',
29 |     device: 'MacBook Air M2',
30 |     type: 'laptop',
31 |     signal: 'good',
32 |     status: 'available',
33 |     pages: 2,
34 |     ip: '192.168.1.78:6190',
35 |   },
36 | ]
37 |
38 | interface ProximitySyncDialogProps {
39 |   onClose: () => void
40 | }
41 |
42 | export function ProximitySyncDialog({ onClose }: ProximitySyncDialogProps) {
43 |   const [enabled, setEnabled] = useState(true)
44 |   const [peers, setPeers] = useState<Peer[]>(INITIAL_PEERS)
```

```
  const [syncingId, setSyncingId] = useState<string | null>(null)
  const [copiedCode, setCopiedCode] = useState(false)
  const pairCode = '849-204'

  const handleSyncWith = (peerId: string) => {
    setSyncingId(peerId)
    setTimeout(() => {
      setPeers((prev) =>
        prev.map((p) => (p.id === peerId ? { ...p, status: 'synced' } : p))
      )
      setSyncingId(null)
    }, 1200)
  }

  const handleCopyCode = () => {
    navigator.clipboard?.writeText(pairCode)
    setCopiedCode(true)
    setTimeout(() => setCopiedCode(false), 2000)
  }

  return (
    <div
      style={{
```

Linii importante: 168-260

```
            onClick={() => setEnabled(!enabled)}
          >
            {enabled ? 'Active' : 'Turn On'}
          </button>
        </div>

        {/* Content */}
        <div style={{ padding: '20px 24px', display: 'flex', flexDirection: 'column', gap: 18 }}>
          {/* Nearby peers list */}
          <div>
            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center',
marginBottom: 10 }}>
              <span style={{ fontSize: 12, fontWeight: 600, color: 'var(--color-text-tertiary)',
textTransform: 'uppercase', letterSpacing: '0.05em' }}>
                Discovered Peers ({peers.length})
              </span>
              <button
                className="btn btn-xs btn-ghost"
                style={{ fontSize: 11 }}
                onClick={() => {}}
              >
                <RefreshCw size={11} /> Rescan
              </button>
            </div>
```

```
190 |
191 |             <div style={{ display: 'flex', flexDirection: 'column', gap: 8 }}>
192 |               {peers.map((peer) => {
193 |                 const isSyncing = syncingId === peer.id
194 |                 return (
195 |                   <div
196 |                     key={peer.id}
197 |                     style={{
198 |                       padding: 12,
199 |                       borderRadius: 10,
200 |                       border: '1px solid var(--color-border-subtle)',
201 |                       background: 'var(--color-bg-subtle)',
202 |                       display: 'flex',
203 |                       alignItems: 'center',
204 |                       justifyContent: 'space-between',
205 |                     }}
206 |                   >
207 |                     <div className="flex items-center gap-3">
208 |                       <div
209 |                         style={{
210 |                           width: 34,
211 |                           height: 34,
212 |                           borderRadius: 8,
213 |                           background: 'var(--color-bg-page)',
214 |                           border: '1px solid var(--color-border-default)',
215 |                           display: 'flex',
216 |                           alignItems: 'center',
217 |                           justifyContent: 'center',
218 |                           color: 'var(--color-brand-600)',
219 |                         }}
220 |                       >
221 |                         {peer.type === 'tablet' ? <Smartphone size={16} /> : <Laptop size={16} />}
222 |                       </div>
223 |                       <div>
224 |                         <div style={{ display: 'flex', alignItems: 'center', gap: 6 }}>
225 |                           <span style={{ fontSize: 13, fontWeight: 600, color: 'var(--color-text-primary)'
}}>
226 |                             {peer.name}
227 |                           </span>
228 |                           <span style={{ fontSize: 11, color: 'var(--color-text-tertiary)' }}>
229 |                             ({peer.device})
230 |                           </span>
231 |                         </div>
232 |                         <div style={{ fontSize: 11, color: 'var(--color-text-tertiary)', display: 'flex',
gap: 10, marginTop: 2 }}>
233 |                           <span>Signal: {peer.signal}</span>
234 |                           <span>•</span>
235 |                           <span>{peer.pages} shared notes</span>
236 |                         </div>
237 |                       </div>
```

```
238 |                     </div>
239 |
240 |                     <div>
241 |                       {peer.status === 'synced' && !isSyncing ? (
242 |                         <span className="badge badge-success flex items-center gap-1" style={{ fontSize: 11
}}>
243 |                           <Check size={11} /> Synced
244 |                         </span>
245 |                       ) : (
246 |                         <button
247 |                           className="btn btn-xs btn-primary flex items-center gap-1"
248 |                           disabled={isSyncing}
249 |                           onClick={() => handleSyncWith(peer.id)}
250 |                         >
251 |                           {isSyncing ? (
252 |                             <>
253 |                               <RefreshCw size={11} className="animate-spin" /> Syncing…
254 |                             </>
255 |                           ) : (
256 |                             'Sync Now'
257 |                           )}
258 |                         </button>
259 |                       )}
260 |                     </div>
```

Linii importante: 300-340

```
300 |                   padding: '4px 8px',
301 |                   borderRadius: 6,
302 |                 }}
303 |               >
304 |                 {pairCode}
305 |               </span>
306 |               <button
307 |                 className="btn btn-icon btn-ghost btn-xs"
308 |                 onClick={handleCopyCode}
309 |                 title="Copy pair code"
310 |               >
311 |                 {copiedCode ? <Check size={14} color="#12B76A" /> : <Copy size={14} />}
312 |               </button>
313 |             </div>
314 |           </div>
315 |
316 |           {/* E2E Security notice */}
317 |           <div className="flex items-center gap-2" style={{ color: 'var(--color-text-tertiary)', fontSize: 11
}}>
318 |             <Shield size={13} style={{ flexShrink: 0, color: '#12B76A' }} />
319 |             <span>End-to-end encrypted direct WebRTC channel. No data leaves your local network.</span>
320 |           </div>
321 |         </div>
```

```
322 |
323 |         {/* Footer */}
324 |         <div
325 |           style={{
326 |             padding: '12px 24px',
327 |             background: 'var(--color-bg-subtle)',
328 |             borderTop: '1px solid var(--color-border-subtle)',
329 |             display: 'flex',
330 |             justifyContent: 'flex-end',
331 |           }}
332 |         >
333 |           <button className="btn btn-secondary btn-sm" onClick={onClose}>
334 |             Done
335 |           </button>
336 |         </div>
337 |       </div>
338 |     </div>
339 |   )
340 | }
```

apps/web/src/components/SearchDialog.tsx

Căutare globală demo. Filtrează rezultate locale din notebook-uri și navighează spre pagină. Pentru milestone-ul backend trebuie înlocuită cu full-text search în Postgres și ulterior semantic search.

Linii importante: 1-62

```
 1 | import { useState, useEffect, useRef } from 'react'
 2 | import { Search, FileText, GitBranch, ArrowRight, X } from 'lucide-react'
 3 | import { useAppStore } from '../store/appStore'
 4 |
 5 | interface SearchResult {
 6 |   id: string
 7 |   title: string
 8 |   type: 'page' | 'notebook'
 9 |   notebookName?: string
10 |   preview?: string
11 | }
12 |
13 | export function SearchDialog() {
14 |   const { notebooks, setSearchOpen, setCurrentPage, setCurrentNotebook } = useAppStore()
15 |   const [query, setQuery] = useState('')
16 |   const [activeIndex, setActiveIndex] = useState(0)
17 |   const inputRef = useRef<HTMLInputElement>(null)
18 |
19 |   useEffect(() => { inputRef.current?.focus() }, [])
20 |
21 |   const allResults: SearchResult[] = [
22 |     ...notebooks.flatMap((nb) =>
23 |       nb.pages.map((p): SearchResult => ({
24 |         id: p.id,
25 |         title: p.title,
26 |         type: 'page',
27 |         notebookName: nb.name,
28 |         preview: 'Click to open this page…',
29 |       }))
30 |     ),
31 |     ...notebooks.map((nb): SearchResult => ({
32 |       id: nb.id,
33 |       title: nb.name,
34 |       type: 'notebook',
35 |     })),
36 |   ]
37 |
38 |   const results = query
39 |     ? allResults.filter((r) =>
40 |         r.title.toLowerCase().includes(query.toLowerCase()) ||
41 |         r.notebookName?.toLowerCase().includes(query.toLowerCase())
42 |       )
43 |     : allResults.slice(0, 8)
44 |
```

```
  45 |   function handleSelect(result: SearchResult) {
  46 |     if (result.type === 'page') {
  47 |       setCurrentPage(result.id)
  48 |       // Find notebook
  49 |       const nb = notebooks.find((n) => n.pages.some((p) => p.id === result.id))
  50 |       if (nb) setCurrentNotebook(nb.id)
  51 |     }
  52 |     setSearchOpen(false)
  53 |   }
  54 |
  55 |   function handleKeyDown(e: React.KeyboardEvent) {
  56 |     if (e.key === 'ArrowDown') { e.preventDefault(); setActiveIndex((i) => (i + 1) % results.length) }
  57 |     if (e.key === 'ArrowUp') { e.preventDefault(); setActiveIndex((i) => (i - 1 + results.length) %
results.length) }
  58 |     if (e.key === 'Enter') { handleSelect(results[activeIndex]); }
  59 |     if (e.key === 'Escape') { setSearchOpen(false) }
  60 |   }
  61 |
  62 |   return (
```

Linii importante: 63-143

```
  63 |     <div
  64 |       className="search-overlay"
  65 |       onClick={(e) => { if (e.target === e.currentTarget) setSearchOpen(false) }}
  66 |       role="dialog"
  67 |       aria-label="Global search"
  68 |     >
  69 |       <div className="search-dialog">
  70 |         <div className="search-input-wrapper">
  71 |           <Search size={16} style={{ color: 'var(--color-text-tertiary)', flexShrink: 0 }} />
  72 |           <input
  73 |             ref={inputRef}
  74 |             className="search-input"
  75 |             placeholder="Search pages, notebooks…"
  76 |             value={query}
  77 |             onChange={(e) => { setQuery(e.target.value); setActiveIndex(0) }}
  78 |             onKeyDown={handleKeyDown}
  79 |             aria-label="Search"
  80 |           />
  81 |           {query && (
  82 |             <button
  83 |               className="btn btn-icon btn-ghost"
  84 |               onClick={() => setQuery('')}
  85 |               style={{ width: 24, height: 24 }}
  86 |               aria-label="Clear search"
  87 |             >
  88 |               <X size={13} />
  89 |             </button>
  90 |           )}
```

```
 91 |           <span style={{ fontSize: 11, color: 'var(--color-text-tertiary)', flexShrink: 0, marginLeft: 4
}}>Esc</span>
 92 |         </div>
 93 |
 94 |         <div className="search-results" role="listbox">
 95 |           {results.length === 0 ? (
 96 |             <div style={{ padding: '24px 16px', textAlign: 'center', color: 'var(--color-text-tertiary)',
fontSize: 13 }}>
 97 |               No results for "{query}"
 98 |             </div>
 99 |           ) : (
100 |             results.map((result, idx) => (
101 |               <div
102 |                 key={result.id}
103 |                 className={`search-result-item ${activeIndex === idx ? 'active' : ''}`}
104 |                 role="option"
105 |                 aria-selected={activeIndex === idx}
106 |                 onMouseEnter={() => setActiveIndex(idx)}
107 |                 onClick={() => handleSelect(result)}
108 |               >
109 |                 {result.type === 'page'
110 |                   ? <FileText size={15} style={{ color: 'var(--color-text-tertiary)', flexShrink: 0 }} />
111 |                   : <GitBranch size={15} style={{ color: 'var(--color-accent)', flexShrink: 0 }} />
112 |                 }
113 |                 <div style={{ flex: 1, overflow: 'hidden' }}>
114 |                   <div style={{ fontSize: 14, fontWeight: 500, color: 'var(--color-text-primary)', overflow:
'hidden', textOverflow: 'ellipsis', whiteSpace: 'nowrap' }}>
115 |                     {result.title}
116 |                   </div>
117 |                   {result.notebookName && (
118 |                     <div style={{ fontSize: 12, color: 'var(--color-text-tertiary)' }}>
119 |                       in {result.notebookName}
120 |                     </div>
121 |                   )}
122 |                 </div>
123 |                 <ArrowRight size={13} style={{ color: 'var(--color-text-disabled)', flexShrink: 0 }} />
124 |               </div>
125 |             ))
126 |           )}
127 |         </div>
128 |
129 |         {/* Footer hint */}
130 |         <div style={{
131 |           padding: '8px 16px',
132 |           borderTop: '1px solid var(--color-border-subtle)',
133 |           display: 'flex', gap: 16,
134 |           fontSize: 11, color: 'var(--color-text-tertiary)'
135 |         }}>
136 |           <span>↑ ↓ navigate</span>
137 |           <span>↵ open</span>
```

```
138 |           <span>Esc close</span>
139 |         </div>
140 |       </div>
141 |     </div>
142 |   )
143 | }
```

apps/web/src/pages/TasksPage.tsx

Task management local. Filtrare, completare, adăugare manuală și buton de extracție AI simulată. Modelul viitor este tabela `tasks` alimentată de server.

Linii importante: 1-30

```
 1 | import { useState } from 'react'
 2 | import { useNavigate } from 'react-router-dom'
 3 | import { CheckSquare, Sparkles, Plus, Calendar, Check } from 'lucide-react'
 4 | import { useAppStore } from '../store/appStore'
 5 |
 6 | interface TaskItem {
 7 |   id: string
 8 |   text: string
 9 |   noteTitle: string
10 |   pageId: string
11 |   notebook: string
12 |   done: boolean
13 |   dueDate?: string
14 |   priority: 'high' | 'medium' | 'low'
15 | }
16 |
17 | const INITIAL_TASKS: TaskItem[] = [
18 |   {
19 |     id: 't-1',
20 |     text: 'Review Routh-Hurwitz stability conditions and polynomial coefficients',
21 |     noteTitle: 'Feedback Stability',
22 |     pageId: 'pg-1',
23 |     notebook: 'Sisteme Automate',
24 |     done: true,
25 |     dueDate: 'Oct 8, 2026',
26 |     priority: 'high',
27 |   },
28 |   {
29 |     id: 't-2',
30 |     text: 'Simulate closed-loop root locus trajectory in MATLAB & verify pole crossings',
```

Linii importante: 70-129

```
70 | export function TasksPage() {
71 |   const navigate = useNavigate()
72 |   const { setCurrentPage } = useAppStore()
73 |   const [tasks, setTasks] = useState<TaskItem[]>(INITIAL_TASKS)
74 |   const [filter, setFilter] = useState<'all' | 'pending' | 'completed'>('all')
75 |   const [extracting, setExtracting] = useState(false)
76 |   const [newTaskText, setNewTaskText] = useState('')
77 |
78 |   const toggleTask = (id: string) => {
79 |     setTasks((prev) =>
80 |       prev.map((t) => (t.id === id ? { ...t, done: !t.done } : t))
81 |     )
```

```
  }

  const handleExtractWithAI = () => {
    setExtracting(true)
    setTimeout(() => {
      setTasks((prev) => [
        {
          id: `t-${Date.now()}`,
          text: 'Derive gain margin and phase margin formulas from Bode plot asymptotes',
          noteTitle: 'Bode Plots',
          pageId: 'pg-1',
          notebook: 'Sisteme Automate',
          done: false,
          dueDate: 'Nov 10, 2026',
          priority: 'medium',
        },
        ...prev,
      ])
      setExtracting(false)
    }, 1200)
  }

  const handleAddTask = (e: React.FormEvent) => {
    e.preventDefault()
    if (!newTaskText.trim()) return
    setTasks((prev) => [
      {
        id: `t-${Date.now()}`,
        text: newTaskText.trim(),
        noteTitle: 'Feedback Stability',
        pageId: 'pg-1',
        notebook: 'Sisteme Automate',
        done: false,
        dueDate: 'Soon',
        priority: 'medium',
      },
      ...prev,
    ])
    setNewTaskText('')
  }

  const filteredTasks = tasks.filter((t) => {
    if (filter === 'pending') return !t.done
    if (filter === 'completed') return t.done
    return true
  })

  const pendingCount = tasks.filter((t) => !t.done).length
```

Linii importante: 168-309

```
168 |             </div>
169 |             <div style={{ fontSize: 12, color: 'var(--color-text-tertiary)' }}>
170 |               Action items extracted from your study notes and lab journals
171 |             </div>
172 |           </div>
173 |         </div>
174 |
175 |         {/* AI Extraction Button */}
176 |         <button
177 |           className="btn btn-sm btn-primary flex items-center gap-2"
178 |           onClick={handleExtractWithAI}
179 |           disabled={extracting}
180 |         >
181 |           <Sparkles size={14} />
182 |           {extracting ? 'Scanning notes with AI…' : 'Extract tasks from notes'}
183 |         </button>
184 |       </header>
185 |
186 |       {/* Content */}
187 |       <div style={{ flex: 1, padding: '32px 48px', maxWidth: 840, width: '100%', margin: '0 auto', overflowY:
'auto' }}>
188 |         {/* Quick Add Bar */}
189 |         <form onSubmit={handleAddTask} style={{ marginBottom: 24, display: 'flex', gap: 10 }}>
190 |           <input
191 |             type="text"
192 |             className="input"
193 |             placeholder="Add a new action item…"
194 |             value={newTaskText}
195 |             onChange={(e) => setNewTaskText(e.target.value)}
196 |             style={{ flex: 1 }}
197 |           />
198 |           <button type="submit" className="btn btn-secondary">
199 |             <Plus size={14} /> Add Task
200 |           </button>
201 |         </form>
202 |
203 |         {/* Filter tabs */}
204 |         <div style={{ display: 'flex', alignItems: 'center', justifyContent: 'space-between', marginBottom:
16 }}>
205 |           <div style={{ display: 'flex', gap: 6 }}>
206 |             {(['all', 'pending', 'completed'] as const).map((mode) => (
207 |               <button
208 |                 key={mode}
209 |                 className={`btn btn-xs ${filter === mode ? 'btn-primary' : 'btn-ghost'}`}
210 |                 onClick={() => setFilter(mode)}
211 |               >
212 |                 {mode.charAt(0).toUpperCase() + mode.slice(1)}
213 |               </button>
214 |             ))}
```

```
215 |           </div>
216 |
217 |           <span style={{ fontSize: 12, color: 'var(--color-text-tertiary)' }}>
218 |             Showing {filteredTasks.length} tasks
219 |           </span>
220 |         </div>
221 |
222 |         {/* Tasks List */}
223 |         <div style={{ display: 'flex', flexDirection: 'column', gap: 8 }}>
224 |           {filteredTasks.map((task) => (
225 |             <div
226 |               key={task.id}
227 |               style={{
228 |                 padding: '14px 18px',
229 |                 borderRadius: 12,
230 |                 border: '1px solid var(--color-border-subtle)',
231 |                 background: task.done ? 'var(--color-bg-subtle)' : 'var(--color-bg-page)',
232 |                 display: 'flex',
233 |                 alignItems: 'center',
234 |                 gap: 14,
235 |                 boxShadow: task.done ? 'none' : '0 1px 3px rgba(0,0,0,0.04)',
236 |                 transition: 'all 0.15s ease',
237 |               }}
238 |             >
239 |               {/* Checkbox */}
240 |               <button
241 |                 onClick={() => toggleTask(task.id)}
242 |                 style={{
243 |                   width: 20,
244 |                   height: 20,
245 |                   borderRadius: 6,
246 |                   border: task.done ? 'none' : '2px solid var(--color-border-strong)',
247 |                   background: task.done ? 'var(--color-brand-600)' : 'transparent',
248 |                   display: 'flex',
249 |                   alignItems: 'center',
250 |                   justifyContent: 'center',
251 |                   cursor: 'pointer',
252 |                   color: 'white',
253 |                   flexShrink: 0,
254 |                   transition: 'background 0.12s',
255 |                 }}
256 |                 aria-label={task.done ? 'Mark pending' : 'Mark done'}
257 |               >
258 |                 {task.done && <Check size={13} strokeWidth={3} />}
259 |               </button>
260 |
261 |               {/* Text & note info */}
262 |               <div style={{ flex: 1, minWidth: 0 }}>
263 |                 <div
264 |                   style={{
```

```
265 |                     fontSize: 14,
266 |                     color: task.done ? 'var(--color-text-tertiary)' : 'var(--color-text-primary)',
267 |                     textDecoration: task.done ? 'line-through' : 'none',
268 |                     fontWeight: task.done ? 400 : 500,
269 |                   }}
270 |                 >
271 |                   {task.text}
272 |                 </div>
273 |                 <div style={{ display: 'flex', alignItems: 'center', gap: 10, marginTop: 4, fontSize: 11,
color: 'var(--color-text-tertiary)' }}>
274 |                   <span
275 |                     style={{ cursor: 'pointer', color: 'var(--color-brand-600)', fontWeight: 500 }}
276 |                     onClick={() => {
277 |                       setCurrentPage(task.pageId)
278 |                       navigate('/')
279 |                     }}
280 |                   >
281 |                     📄 {task.noteTitle}
282 |                   </span>
283 |                   <span>•</span>
284 |                   <span>📁 {task.notebook}</span>
285 |                   {task.dueDate && (
286 |                     <>
287 |                       <span>•</span>
288 |                       <span className="flex items-center gap-1">
289 |                         <Calendar size={11} /> {task.dueDate}
290 |                       </span>
291 |                     </>
292 |                   )}
293 |                 </div>
294 |               </div>
295 |
296 |               {/* Priority badge */}
297 |               <span
298 |                 className={`badge ${task.priority === 'high' ? 'badge-error' : task.priority === 'medium' ?
'badge-warning' : 'badge-secondary'}`}
299 |                 style={{ fontSize: 10, textTransform: 'uppercase' }}
300 |               >
301 |                 {task.priority}
302 |               </span>
303 |             </div>
304 |           ))}
305 |         </div>
306 |       </div>
307 |     </div>
308 |   )
309 | }
```

apps/web/src/pages/KnowledgeGraphPage.tsx

Knowledge graph demo. Construiește noduri și muchii din date statice, permite filtrare, zoom și deschiderea unei note. În arhitectura țintă, muchiile ar proveni din `page_links` și rezultatele AI.

Linii importante: 1-32

```
 1 | import { useState, useMemo } from 'react'
 2 | import { useNavigate } from 'react-router-dom'
 3 | import { GitBranch, Search, ZoomIn, ZoomOut, RotateCcw, BookOpen, Sparkles } from 'lucide-react'
 4 | import { useAppStore } from '../store/appStore'
 5 |
 6 | interface GraphNode {
 7 |   id: string
 8 |   pageId: string
 9 |   title: string
10 |   notebook: string
11 |   emoji: string
12 |   x: number
13 |   y: number
14 |   radius: number
15 |   color: string
16 |   connections: string[]
17 |   tags: string[]
18 | }
19 |
20 | const GRAPH_NODES: GraphNode[] = [
21 |   {
22 |     id: 'n-1',
23 |     pageId: 'pg-1',
24 |     title: 'Feedback Stability',
25 |     notebook: 'Sisteme Automate',
26 |     emoji: '📊',
27 |     x: 400,
28 |     y: 280,
29 |     radius: 36,
30 |     color: '#7F56D9',
31 |     connections: ['n-2', 'n-3', 'n-4', 'n-5'],
32 |     tags: ['Control', 'Stability', 'LTI'],
```

Linii importante: 114-161

```
114 | export function KnowledgeGraphPage() {
115 |   const navigate = useNavigate()
116 |   const { setCurrentPage, setCurrentNotebook } = useAppStore()
117 |   const [search, setSearch] = useState('')
118 |   const [selectedNotebook, setSelectedNotebook] = useState<string>('all')
119 |   const [selectedNodeId, setSelectedNodeId] = useState<string>('n-1')
120 |   const [zoom, setZoom] = useState(1)
121 |
122 |   const selectedNode = GRAPH_NODES.find((n) => n.id === selectedNodeId)
123 |
```

```
124 |   const filteredNodes = useMemo(() => {
125 |     return GRAPH_NODES.filter((node) => {
126 |       const matchSearch =
127 |         !search ||
128 |         node.title.toLowerCase().includes(search.toLowerCase()) ||
129 |         node.tags.some((t) => t.toLowerCase().includes(search.toLowerCase()))
130 |       const matchNotebook =
131 |         selectedNotebook === 'all' || node.notebook === selectedNotebook
132 |       return matchSearch && matchNotebook
133 |     })
134 |   }, [search, selectedNotebook])
135 |
136 |   const edges = useMemo(() => {
137 |     const list: { from: GraphNode; to: GraphNode }[] = []
138 |     const seen = new Set<string>()
139 |
140 |     GRAPH_NODES.forEach((node) => {
141 |       node.connections.forEach((targetId) => {
142 |         const target = GRAPH_NODES.find((n) => n.id === targetId)
143 |         if (target) {
144 |           const key = [node.id, target.id].sort().join('-')
145 |           if (!seen.has(key)) {
146 |             seen.add(key)
147 |             list.push({ from: node, to: target })
148 |           }
149 |         }
150 |       })
151 |     })
152 |     return list
153 |   }, [])
154 |
155 |   const handleOpenNote = (pageId: string) => {
156 |     setCurrentPage(pageId)
157 |     setCurrentNotebook('nb-1')
158 |     navigate('/')
159 |   }
160 |
161 |   return (
```

Linii importante: 162-502

```
162 |     <div style={{ display: 'flex', flexDirection: 'column', height: '100%', background: 'var(--color-bg-
subtle)' }}>
163 |       {/* Top bar */}
164 |       <header
165 |         className="topbar"
166 |         style={{
167 |           borderBottom: '1px solid var(--color-border-subtle)',
168 |           padding: '12px 24px',
169 |           background: 'var(--color-bg-page)',
```

```
170 |           display: 'flex',
171 |           alignItems: 'center',
172 |           justifyContent: 'space-between',
173 |         }}
174 |       >
175 |         <div className="flex items-center gap-3">
176 |           <div
177 |             style={{
178 |               width: 34,
179 |               height: 34,
180 |               borderRadius: 8,
181 |               background: 'var(--color-brand-100)',
182 |               color: 'var(--color-brand-700)',
183 |               display: 'flex',
184 |               alignItems: 'center',
185 |               justifyContent: 'center',
186 |             }}
187 |           >
188 |             <GitBranch size={18} />
189 |           </div>
190 |           <div>
191 |             <div style={{ display: 'flex', alignItems: 'center', gap: 8 }}>
192 |               <h1 style={{ fontSize: 16, fontWeight: 600, color: 'var(--color-text-primary)' }}>
193 |                 Knowledge Graph
194 |               </h1>
195 |               <span className="badge badge-brand" style={{ fontSize: 11 }}>
196 |                 {GRAPH_NODES.length} notes connected
197 |               </span>
198 |             </div>
199 |             <div style={{ fontSize: 12, color: 'var(--color-text-tertiary)' }}>
200 |               Explore semantic connections, references, and prerequisites
201 |             </div>
202 |           </div>
203 |         </div>
204 |
205 |         {/* Filter controls */}
206 |         <div className="flex items-center gap-3">
207 |           {/* Search */}
208 |           <div
209 |             style={{
210 |               display: 'flex',
211 |               alignItems: 'center',
212 |               gap: 8,
213 |               padding: '6px 12px',
214 |               borderRadius: 8,
215 |               border: '1px solid var(--color-border-default)',
216 |               background: 'var(--color-bg-page)',
217 |               width: 220,
218 |             }}
219 |           >
```

```
            <Search size={14} style={{ color: 'var(--color-text-tertiary)' }} />
            <input
              type="text"
              placeholder="Search concepts or tags…"
              value={search}
              onChange={(e) => setSearch(e.target.value)}
              style={{
                border: 'none',
                outline: 'none',
                background: 'transparent',
                fontSize: 13,
                width: '100%',
                color: 'var(--color-text-primary)',
              }}
            />
          </div>

          {/* Notebook filter */}
          <div className="flex items-center gap-1">
            {['all', 'Sisteme Automate', 'Programare Avansată'].map((nb) => (
              <button
                key={nb}
                className={`btn btn-xs ${selectedNotebook === nb ? 'btn-primary' : 'btn-secondary'}`}
                onClick={() => setSelectedNotebook(nb)}
              >
                {nb === 'all' ? 'All' : nb}
              </button>
            ))}
          </div>

          {/* Zoom */}
          <div
            style={{
              display: 'flex',
              alignItems: 'center',
              border: '1px solid var(--color-border-default)',
              borderRadius: 8,
              background: 'var(--color-bg-page)',
            }}
          >
            <button
              className="btn btn-icon btn-ghost btn-xs"
              onClick={() => setZoom((z) => Math.min(z + 0.15, 1.6))}
              title="Zoom In"
            >
              <ZoomIn size={14} />
            </button>
            <button
              className="btn btn-icon btn-ghost btn-xs"
              onClick={() => setZoom((z) => Math.max(z - 0.15, 0.6))}
```

```
              title="Zoom Out"
            >
              <ZoomOut size={14} />
            </button>
            <button
              className="btn btn-icon btn-ghost btn-xs"
              onClick={() => setZoom(1)}
              title="Reset Zoom"
            >
              <RotateCcw size={13} />
            </button>
          </div>
        </div>
      </header>

      {/* Main Graph Canvas and Side Inspector */}
      <div style={{ flex: 1, display: 'flex', position: 'relative', overflow: 'hidden' }}>
        {/* SVG Graph View */}
        <div style={{ flex: 1, position: 'relative', overflow: 'hidden', cursor: 'grab' }}>
          <svg
            style={{
              width: '100%',
              height: '100%',
              transform: `scale(${zoom})`,
              transformOrigin: 'center center',
              transition: 'transform 0.15s ease-out',
            }}
            viewBox="0 0 960 640"
          >
            {/* Grid Pattern */}
            <defs>
              <pattern id="graph-grid" width="40" height="40" patternUnits="userSpaceOnUse">
                <path d="M 40 0 L 0 0 0 40" fill="none" stroke="rgba(0,0,0,0.04)" strokeWidth="1" />
              </pattern>
            </defs>
            <rect width="960" height="640" fill="url(#graph-grid)" />

            {/* Edges */}
            {edges.map(({ from, to }, idx) => {
              const isSelected = selectedNode && (from.id === selectedNode.id || to.id === selectedNode.id)
              return (
                <line
                  key={idx}
                  x1={from.x}
                  y1={from.y}
                  x2={to.x}
                  y2={to.y}
                  stroke={isSelected ? 'var(--color-brand-600)' : 'var(--color-border-strong)'}
                  strokeWidth={isSelected ? 2.5 : 1.2}
                  strokeDasharray={isSelected ? 'none' : '4,4'}
```

```
320 |                   opacity={isSelected ? 0.9 : 0.4}
321 |                 />
322 |               )
323 |             })}
324 |
325 |             {/* Nodes */}
326 |             {filteredNodes.map((node) => {
327 |               const isSelected = selectedNodeId === node.id
328 |               return (
329 |                 <g
330 |                   key={node.id}
331 |                   onClick={() => setSelectedNodeId(node.id)}
332 |                   style={{ cursor: 'pointer', transition: 'all 0.2s' }}
333 |                 >
334 |                   {/* Selection glow */}
335 |                   {isSelected && (
336 |                     <circle
337 |                       cx={node.x}
338 |                       cy={node.y}
339 |                       r={node.radius + 8}
340 |                       fill="none"
341 |                       stroke="var(--color-brand-500)"
342 |                       strokeWidth="3"
343 |                       opacity="0.35"
344 |                     />
345 |                   )}
346 |
347 |                   {/* Main circle */}
348 |                   <circle
349 |                     cx={node.x}
350 |                     cy={node.y}
351 |                     r={node.radius}
352 |                     fill={isSelected ? node.color : 'var(--color-bg-page)'}
353 |                     stroke={node.color}
354 |                     strokeWidth={isSelected ? 3 : 2}
355 |                     filter="drop-shadow(0 2px 6px rgba(0,0,0,0.08))"
356 |                   />
357 |
358 |                   {/* Emoji / Icon */}
359 |                   <text
360 |                     x={node.x}
361 |                     y={node.y + 6}
362 |                     textAnchor="middle"
363 |                     fontSize={node.radius > 30 ? 20 : 16}
364 |                     style={{ pointerEvents: 'none', userSelect: 'none' }}
365 |                   >
366 |                     {node.emoji}
367 |                   </text>
368 |
369 |                   {/* Node label */}
```

```
370 |                   <text
371 |                     x={node.x}
372 |                     y={node.y + node.radius + 16}
373 |                     textAnchor="middle"
374 |                     fontSize="12"
375 |                     fontWeight={isSelected ? '600' : '500'}
376 |                     fill="var(--color-text-primary)"
377 |                     style={{ pointerEvents: 'none', userSelect: 'none' }}
378 |                   >
379 |                     {node.title}
380 |                   </text>
381 |                 </g>
382 |               )
383 |             })}
384 |           </svg>
385 |         </div>
386 |
387 |         {/* Selected Node Details Drawer */}
388 |         {selectedNode && (
389 |           <aside
390 |             style={{
391 |               width: 340,
392 |               background: 'var(--color-bg-page)',
393 |               borderLeft: '1px solid var(--color-border-default)',
394 |               boxShadow: 'var(--shadow-lg)',
395 |               display: 'flex',
396 |               flexDirection: 'column',
397 |               zIndex: 10,
398 |             }}
399 |           >
400 |             <div style={{ padding: '20px 24px', borderBottom: '1px solid var(--color-border-subtle)' }}>
401 |               <div className="flex items-center gap-3">
402 |                 <span style={{ fontSize: 28 }}>{selectedNode.emoji}</span>
403 |                 <div>
404 |                   <h2 style={{ fontSize: 16, fontWeight: 600, color: 'var(--color-text-primary)' }}>
405 |                     {selectedNode.title}
406 |                   </h2>
407 |                   <div style={{ fontSize: 12, color: 'var(--color-text-tertiary)', marginTop: 2 }}>
408 |                     📁 {selectedNode.notebook}
409 |                   </div>
410 |                 </div>
411 |               </div>
412 |             </div>
413 |
414 |             <div style={{ padding: '20px 24px', flex: 1, overflowY: 'auto', display: 'flex', flexDirection:
'column', gap: 18 }}>
415 |               {/* Tags */}
416 |               <div>
417 |                 <div style={{ fontSize: 11, fontWeight: 600, color: 'var(--color-text-tertiary)',
textTransform: 'uppercase', marginBottom: 8 }}>
```

```
418 |                   Tags & Concepts
419 |                 </div>
420 |                 <div style={{ display: 'flex', flexWrap: 'wrap', gap: 6 }}>
421 |                   {selectedNode.tags.map((tag) => (
422 |                     <span key={tag} className="badge badge-secondary" style={{ fontSize: 11 }}>
423 |                       #{tag}
424 |                     </span>
425 |                   ))}
426 |                 </div>
427 |               </div>
428 |
429 |               {/* Connected notes */}
430 |               <div>
431 |                 <div style={{ fontSize: 11, fontWeight: 600, color: 'var(--color-text-tertiary)',
textTransform: 'uppercase', marginBottom: 8 }}>
432 |                   Linked Notes ({selectedNode.connections.length})
433 |                 </div>
434 |                 <div style={{ display: 'flex', flexDirection: 'column', gap: 6 }}>
435 |                   {selectedNode.connections.map((connId) => {
436 |                     const conn = GRAPH_NODES.find((n) => n.id === connId)
437 |                     if (!conn) return null
438 |                     return (
439 |                       <div
440 |                         key={conn.id}
441 |                         onClick={() => setSelectedNodeId(conn.id)}
442 |                         style={{
443 |                           padding: '8px 12px',
444 |                           borderRadius: 8,
445 |                           background: 'var(--color-bg-subtle)',
446 |                           border: '1px solid var(--color-border-subtle)',
447 |                           display: 'flex',
448 |                           alignItems: 'center',
449 |                           justifyContent: 'space-between',
450 |                           cursor: 'pointer',
451 |                         }}
452 |                       >
453 |                         <div className="flex items-center gap-2">
454 |                           <span>{conn.emoji}</span>
455 |                           <span style={{ fontSize: 12, fontWeight: 500, color: 'var(--color-text-primary)'
}}>
456 |                             {conn.title}
457 |                           </span>
458 |                         </div>
459 |                         <span style={{ fontSize: 11, color: 'var(--color-text-tertiary)' }}>
460 |                           {conn.notebook}
461 |                         </span>
462 |                       </div>
463 |                     )
464 |                   })}
465 |                 </div>
```

```
466 |               </div>
467 |
468 |               {/* AI Graph Insight */}
469 |               <div
470 |                 style={{
471 |                   padding: 12,
472 |                   borderRadius: 10,
473 |                   background: 'var(--color-brand-50)',
474 |                   border: '1px solid var(--color-brand-100)',
475 |                 }}
476 |               >
477 |                 <div className="flex items-center gap-2" style={{ color: 'var(--color-brand-700)',
marginBottom: 6 }}>
478 |                   <Sparkles size={14} />
479 |                   <span style={{ fontSize: 12, fontWeight: 600 }}>AI Knowledge Synthesis</span>
480 |                 </div>
481 |                 <p style={{ fontSize: 12, color: 'var(--color-brand-900)', lineHeight: 1.5 }}>
482 |                   This note serves as a bridge between continuous-time control theory and discrete simulation
models.
483 |                 </p>
484 |               </div>
485 |
486 |               {/* Open Note Button */}
487 |               <div style={{ marginTop: 'auto', paddingTop: 16 }}>
488 |                 <button
489 |                   className="btn btn-primary"
490 |                   style={{ width: '100%', justifyContent: 'center' }}
491 |                   onClick={() => handleOpenNote(selectedNode.pageId)}
492 |                 >
493 |                   <BookOpen size={14} /> Open Note
494 |                 </button>
495 |               </div>
496 |             </div>
497 |           </aside>
498 |         )}
499 |       </div>
500 |     </div>
501 |   )
502 | }
```

apps/web/package.json

Dependențe și build. TipTap este editorul, React/Vite platforma, Zustand starea, React Router routing-ul, Lucide iconițele și perfect-freehand desenul. Sunt prezente biblioteci pentru direcția UI, dar nu sunt încă prezente Yjs, Hocuspocus, Supabase sau un server Node.

Linii importante: 1-71

```
 1 | {
 2 |   "name": "web",
 3 |   "private": true,
 4 |   "version": "0.1.0",
 5 |   "type": "module",
 6 |   "scripts": {
 7 |     "dev": "vite",
 8 |     "build": "tsc -b && vite build",
 9 |     "lint": "eslint .",
10 |     "preview": "vite preview",
11 |     "test": "vitest"
12 |   },
13 |   "dependencies": {
14 |     "@tiptap/core": "^2.10.3",
15 |     "@tiptap/extension-blockquote": "^2.10.3",
16 |     "@tiptap/extension-bold": "^2.10.3",
17 |     "@tiptap/extension-bullet-list": "^2.10.3",
18 |     "@tiptap/extension-code": "^2.10.3",
19 |     "@tiptap/extension-code-block": "^2.10.3",
20 |     "@tiptap/extension-color": "^2.10.3",
21 |     "@tiptap/extension-document": "^2.10.3",
22 |     "@tiptap/extension-dropcursor": "^2.10.3",
23 |     "@tiptap/extension-gapcursor": "^2.10.3",
24 |     "@tiptap/extension-hard-break": "^2.10.3",
25 |     "@tiptap/extension-heading": "^2.10.3",
26 |     "@tiptap/extension-highlight": "^2.10.3",
27 |     "@tiptap/extension-history": "^2.10.3",
28 |     "@tiptap/extension-horizontal-rule": "^2.10.3",
29 |     "@tiptap/extension-image": "^2.10.3",
30 |     "@tiptap/extension-italic": "^2.10.3",
31 |     "@tiptap/extension-link": "^2.10.3",
32 |     "@tiptap/extension-list-item": "^2.10.3",
33 |     "@tiptap/extension-ordered-list": "^2.10.3",
34 |     "@tiptap/extension-paragraph": "^2.10.3",
35 |     "@tiptap/extension-placeholder": "^2.10.3",
36 |     "@tiptap/extension-strike": "^2.10.3",
37 |     "@tiptap/extension-table": "^2.10.3",
38 |     "@tiptap/extension-table-cell": "^2.10.3",
39 |     "@tiptap/extension-table-header": "^2.10.3",
40 |     "@tiptap/extension-table-row": "^2.10.3",
41 |     "@tiptap/extension-task-item": "^2.10.3",
42 |     "@tiptap/extension-task-list": "^2.10.3",
43 |     "@tiptap/extension-text": "^2.10.3",
```

```
    "@tiptap/extension-text-style": "^2.10.3",
    "@tiptap/extension-underline": "^2.10.3",
    "@tiptap/pm": "^2.10.3",
    "@tiptap/react": "^2.10.3",
    "@tiptap/starter-kit": "^2.10.3",
    "clsx": "^2.1.1",
    "lucide-react": "^0.468.0",
    "perfect-freehand": "^1.2.2",
    "react": "^19.0.0",
    "react-dom": "^19.0.0",
    "react-router-dom": "^7.1.1",
    "zustand": "^5.0.3"
  },
  "devDependencies": {
    "@eslint/js": "^9.17.0",
    "@types/node": "^26.6.5",
    "@types/react": "^19.0.2",
    "@types/react-dom": "^19.0.2",
    "@vitejs/plugin-react": "^4.3.4",
    "eslint": "^9.17.0",
    "eslint-plugin-react-hooks": "^5.0.0",
    "eslint-plugin-react-refresh": "^0.4.16",
    "globals": "^15.14.0",
    "typescript": "~5.7.2",
    "vite": "^6.0.5",
    "vitest": "^2.1.8"
  }
}
```

SECOND_BRAIN_IMPLEMENTATION.md

Documentul de arhitectură și plan. Definește teza: sync realtime caracter-cu-caracter și stroke-cu-stroke, offline-first/P2P și AI/RAG cu citări. Propune monorepo cu `apps/web`, `apps/server`, `packages/*`, Supabase/Postgres/pgvector, Hocuspocus și Yjs.

Linii importante: 1-42

```
   1 | # Second Brain: Implementation Plan and Figma Handoff
   2 |
   3 | A note-taking app with block-based text, OneNote-style drawing, instant sync, offline peer-to-peer, AI over
your own notes, and a knowledge graph. Bachelor's thesis project (ACS, UNSTPB).
   4 |
   5 | How to use this file: put it in the repo (e.g. `docs/IMPLEMENTATION.md`) and copy section 9 into `CLAUDE.md`
at the repo root so Claude Code reads it every session.
   6 |
   7 | ---
   8 |
   9 | ## 1. Goal and scope
  10 |
  11 | **Product:** desktop-first (web/PWA, optional Tauri), with tablet and mobile layouts.
  12 |
  13 | **Thesis argument (the three things that must work well):**
  14 | 1. **Real-time sync** of text (character level) and drawings (stroke level) using CRDTs (Yjs).
  15 | 2. **Offline-first + P2P**: edit with no internet, merge without conflicts, sync directly between nearby
devices.
  16 | 3. **AI on your own notes** (RAG with source citations), including handwriting-to-text.
  17 |
  18 | **Everything else is secondary** (knowledge graph, tasks, settings, onboarding). Build it simple, or list it
as "future work". It is better to have 3 excellent features than 8 half-working ones.
  19 |
  20 | ### Honest technical caveats (put these in the thesis, not hidden)
  21 | - **WebRTC needs a signaling step** even on a LAN. Truly offline P2P means: one device runs a tiny local
signaling server, or devices exchange connection data by QR code / copy-paste. Plan the P2P demo around that.
  22 | - **Bluetooth LE / Wi-Fi Direct are not available in browsers.** Treat them as "future work" or a native
proof of concept. The reliable demo is WebRTC over the local network.
  23 | - **Tauri mobile** is still maturing. Use a PWA for tablet/mobile and Tauri only for desktop, if time allows.
  24 | - **Handwriting recognition and sketch description need internet** (vision model). Say so in the UI (the
design already does).
  25 |
  26 | ---
  27 |
  28 | ## 2. Recommended stack
  29 |
  30 | | Area | Choice | Notes |
  31 | |---|---|---|
  32 | | Language | TypeScript everywhere | |
  33 | | Monorepo | pnpm workspaces | `apps/web`, `apps/server`, `packages/*` |
  34 | | Web app | React + Vite | PWA via `vite-plugin-pwa` |
  35 | | Styling | Tailwind CSS + CSS variables from design tokens | Untitled UI-like look |
  36 | | Editor | TipTap (ProseMirror) | Collaboration + collaboration caret extensions |
```

```
  37 | | CRDT | Yjs | one `Y.Doc` per page |
  38 | | Sync server | Hocuspocus (Yjs websocket server) | auth + persistence hooks |
  39 | | Local persistence | `y-indexeddb` | offline-first |
  40 | | P2P | `y-webrtc` (or custom WebRTC provider) | needs signaling, see caveats |
  41 | | Drawing | HTML canvas + `perfect-freehand` | stroke data in Yjs; see section 6 |
  42 | | Backend/DB | Supabase (Postgres + Auth + `pgvector`) | simplest path to auth + vectors |
```

Linii importante: 44-91

```
  44 | | AI | Anthropic or OpenAI API (embeddings + chat + vision) | keys only on the server |
  45 | | Graph | `d3-force` or `react-force-graph` | with accessible list view |
  46 | | Tests | Vitest, Playwright (multi-tab sync tests) | |
  47 |
  48 | Alternatives if you prefer to save time: **tldraw** for the whole drawing canvas (it has its own sync story,
but less control for the thesis), or Supabase Realtime instead of Hocuspocus (loses CRDT benefits, not recommended).
  49 |
  50 | ---
  51 |
  52 | ## 3. Repository layout
  53 |
  54 | ```
  55 | second-brain/
  56 |   CLAUDE.md                  # copy of section 9
  57 |   docs/IMPLEMENTATION.md     # this file
  58 |   apps/
  59 |     web/                     # React app
  60 |       src/
  61 |         app/                 # routes, layout shell
  62 |         features/
  63 |           editor/            # TipTap, slash menu, bubble menu, blocks
  64 |           drawing/           # canvas block, palette, strokes
  65 |           copilot/           # AI panel, RAG chat
  66 |           graph/             # knowledge graph
  67 |           search/            # command palette
  68 |           sync/              # providers, status indicators
  69 |           p2p/               # proximity sync
  70 |           auth/ settings/ onboarding/
  71 |         lib/                 # yjs setup, api client, utils
  72 |     server/                  # hocuspocus + API
  73 |   packages/
  74 |     tokens/                  # design tokens (from Figma variables)
  75 |     ui/                      # shared components (button, input, popover...)
  76 |     shared/                  # types shared by web + server
  77 | ```
  78 |
  79 | ---
  80 |
  81 | ## 4. Design source (Figma)
  82 |
  83 | - **File:** `https://www.figma.com/design/tmKb2phvSvpbWMXgcnPjPW/Untitled`
```

```
  84 | - The Figma file is a **specification**, not final code. Implement with real components and tokens; do not copy generated layer structure literally.
  85 | - **Node IDs in this file change** when frames are regenerated. Never hard-code a node ID in docs. Always use *Copy link to selection* on the frame you need, at the moment you need it.
  86 | - Variables: collections `Second Brain • Foundations` and `Second Brain • Semantic` (Light/Dark). Map them to `packages/tokens` (CSS variables) first.
  87 |
  88 | ### Screen map (frame name in Figma → feature)
  89 |
  90 | | Figma frame | Feature | Phase |
  91 | |---|---|---|
```

Linii importante: 93-166

```
  93 | | Editor with slash menu open | slash commands | 3 |
  94 | | Editor blocks • Insertion and selection | block types, bubble menu | 3 |
  95 | | Workspace with AI Co-pilot open | Co-pilot panel | 7 |
  96 | | Drawing mode · pen palette open | canvas block + palette + pen popover | 5 |
  97 | | Drawing toolkit • Component specimens | palette docking, tool and swatch states | 5 |
  98 | | Drawing popovers • Component specimens | per-tool popovers | 5 |
  99 | | Canvas • State board | canvas states (empty, selection, offline...) | 5 |
 100 | | Expanded drawing • Light / Dark | full-screen drawing | 5 |
 101 | | Drawing recognition • AI Co-pilot / AI drawing • State board | handwriting to text, describe sketch | 7 |
 102 | | Knowledge graph • University / State board | graph | 8 |
 103 | | Proximity Sync • Sequence board, Nearby collaboration | P2P | 9 |
 104 | | Global search • Notes and indexed ink / State board | command palette | 6 |
 105 | | Authentication • Login / Register / Recovery / State board | auth | 6 |
 106 | | Onboarding • (3 frames) | first-run flow | 6 |
 107 | | Settings • Account, Sync, AI and Export | settings | 10 |
 108 | | Shared states • Sync, AI and grounded panels | sync/AI status components | 4, 7 |
 109 | | Tablet landscape / portrait, Mobile • (2 frames) | responsive layouts | 11 |
 110 | | Second Brain • Keyboard and access | shortcuts + accessibility rules | 11 |
 111 |
 112 | ### Known design gaps (fix in Figma or decide in code)
 113 | - Palette tool icons are inconsistent between frames (placeholder circles, letter glyphs, or real icons). **In code use Lucide icons:** `mouse-pointer-2`, `pen-line`, `highlighter`, `eraser`, `lasso`, `shapes`, `type`, `undo-2`, `redo-2`, `grip-vertical`, `more-horizontal`.
 114 | - Some newer frames have no sidebar icons or avatar. The first 3 screens (Workspace, Slash menu, Co-pilot) are the visual reference for the shell.
 115 | - No local Figma components exist yet. Treat the toolkit sheet as the spec for states.
 116 | - Slash menu has no "Canvas" item in Figma: add it in code (inserts a canvas block).
 117 |
 118 | ---
 119 |
 120 | ## 5. Data model (starting point)
 121 |
 122 | ### Postgres (Supabase)
 123 | ```sql
 124 | workspaces(id, name, kind /* personal|university|project */, owner_id)
 125 | notebooks(id, workspace_id, name, position)
```

```
 126 | pages(id, notebook_id, parent_page_id, title, icon, cover, updated_at)
 127 | page_snapshots(page_id, ydoc_state bytea, updated_at)   -- Yjs binary state
 128 | page_chunks(id, page_id, block_id, text, embedding vector(1536), updated_at)
 129 | page_links(source_page_id, target_page_id, kind /* manual|ai */, score)
 130 | tasks(id, page_id, text, due_at, done, source_block_id)
 131 | members(workspace_id, user_id, role)
 132 | ```
 133 | - Pages store their content as a **Yjs binary snapshot**, not as JSON blocks. Text extraction for search/RAG
happens server-side after each debounced save.
 134 |
 135 | ### Yjs document (one per page)
 136 | ```
 137 | ydoc
 138 |   ├── "content"   Y.XmlFragment   -> TipTap/ProseMirror document
 139 |   ├── "canvas:<blockId>"  Y.Map   -> one per canvas block (see section 6)
 140 |   └── awareness   (not persisted) -> cursors, names, colors, live strokes
 141 | ```
 142 |
 143 | ---
 144 |
 145 | ## 6. Drawing and ink sync (the OneNote part)
 146 |
 147 | **Canvas block** = a custom TipTap node. It stores only `blockId` and size in the document. The actual
strokes live in a separate Yjs structure keyed by that id, so heavy ink data never bloats the text document.
 148 |
 149 | ```ts
 150 | // committed strokes (persisted + synced via Yjs)
 151 | type Stroke = {
 152 |   id: string; tool: 'pen' | 'highlighter';
 153 |   color: string; width: number; opacity: number;
 154 |   points: [x: number, y: number, pressure: number][];
 155 |   authorId: string; createdAt: number;
 156 | };
 157 | // canvas:<blockId> -> Y.Map<strokeId, Stroke>   (immutable once committed)
 158 | ```
 159 |
 160 | **Rules:**
 161 | - **Live strokes** (while the finger/pen is down) travel through **awareness** (ephemeral, throttled ~30-60
ms). On `pointerup` the finished stroke is written **once** to Yjs. This gives a "collaborator is drawing right now"
effect without flooding the document.
 162 | - **Eraser (stroke mode)** deletes a stroke by id. A pixel eraser is optional (v2): it would split strokes.
 163 | - **Undo/redo:** `Y.UndoManager` with a per-user origin, so Alice's undo never removes Bob's strokes.
 164 | - **Rendering:** `perfect-freehand` generates the outline from points + pressure; draw to canvas. Keep a
"committed" layer and a "live" layer.
 165 | - **Input:** Pointer Events, `touch-action: none` on the canvas, use `pointerType` (`pen` / `touch` /
`mouse`) for palm rejection, and read `pressure`.
 166 | - **Selection / lasso:** hit-test strokes by bounding box, then by point distance. Move/resize edits the
strokes' points (a new transaction).
```

Linii importante: 168-224

```
168 | - **Handwriting to text / describe sketch:** render the selected strokes to a PNG on the client, send it to the server endpoint, which calls a vision model and returns text. Show the result as an editable overlay with accept/reject (see the "Drawing recognition" frames). Always keep the original ink.
169 |
170 | ---
171 |
172 | ## 7. Phased implementation plan
173 |
174 | Each phase ends with a demo you can show and a measurable check. Do not start a phase before the previous one passes its "Done when".
175 |
176 | ### Phase 0: Setup (0.5 day)
177 | - [ ] pnpm monorepo, TypeScript, ESLint/Prettier, Vitest, Playwright, CI (GitHub Actions).
178 | - [ ] `apps/web` (Vite + React + Tailwind), `apps/server` (Node), `packages/shared`.
179 | - [ ] Supabase project (Auth + Postgres + pgvector), `.env.example`.
180 |
181 | **Done when:** `pnpm dev` starts web + server; CI is green on an empty test.
182 |
183 | ### Phase 1: Design tokens + base UI (1-2 days)
184 | - [ ] Pull variables from Figma into `packages/tokens` as CSS variables (Light + Dark).
185 | - [ ] Inter font; text styles (sm/md/lg, weights).
186 | - [ ] Base components: Button, Input, Select, Badge, Avatar, Tooltip, Popover, Dialog, Toggle, Slider, SegmentedControl, KbdHint.
187 |
188 | **Done when:** a Storybook/Ladle page shows each component in all states (default, hover, focus, disabled), in Light and Dark, and it matches Figma.
189 |
190 | ### Phase 2: App shell (1-2 days)
191 | - [ ] 3-column layout: sidebar (workspace switcher, quick search, All notes, Graph, My tasks, notebooks tree, Offline/P2P card, sync status, user), content, right Co-pilot panel (collapsible).
192 | - [ ] Routing (`/workspace/:id/page/:pageId`, `/graph`, `/tasks`, `/settings`).
193 | - [ ] Notebook/page tree with nesting (mock data first).
194 |
195 | **Done when:** the shell matches the "Workspace with editor" frame at 1440 px.
196 |
197 | ### Phase 3: Editor (3-4 days)
198 | - [ ] TipTap with blocks: paragraph, H1-H3, to-do, quote, code block, table, image, nested page link.
199 | - [ ] Slash menu (`/`) with search, groups, keyboard navigation (arrows, Enter, Esc), hint chips.
200 | - [ ] Bubble menu on selection; drag handle per block.
201 | - [ ] Page header: title, icon, cover, "Lecture" chip, date, per-page sync status.
202 |
203 | **Done when:** all blocks can be created via slash menu and keyboard only; slash menu matches the design.
204 |
205 | ### Phase 4: Real-time sync + offline (3-4 days)  **(thesis core)**
206 | - [ ] Hocuspocus server with auth hook (Supabase JWT) and persistence (`page_snapshots`).
207 | - [ ] TipTap Collaboration + collaboration caret (names + colors).
208 | - [ ] `y-indexeddb` for offline persistence.
209 | - [ ] Sync status component with states: **synced, syncing, offline (N changes queued), reconnected and merged**.
```

```
 210 | - [ ] Offline banner and "auto-merged" notice.
 211 |
 212 | **Done when (automated Playwright test):**
 213 | 1. Two browser contexts edit the same page concurrently, and both converge to identical content.
 214 | 2. One context goes offline, makes edits, reconnects, and merges with no lost edits.
 215 | 3. Measure and log round-trip latency (target: < 100-200 ms on localhost; report percentiles in the thesis).
 216 |
 217 | ### Phase 5: Drawing (4-6 days)  **(thesis core)**
 218 | - [ ] Canvas block node + embedded canvas component (empty, drawing, selection, read-only, offline states).
 219 | - [ ] Pen, highlighter, eraser (stroke), lasso/select, shapes, text label.
 220 | - [ ] Palette component: tools, color swatches (+ recent + custom), size chip, undo/redo, more menu, docking
(bottom/top/left/right), collapse to single button.
 221 | - [ ] Per-tool popovers (pen settings first: type, color grid, thickness, opacity, preview).
 222 | - [ ] Live strokes via awareness; committed strokes via Yjs; per-user undo.
 223 | - [ ] Expanded (full-screen) drawing mode.
 224 | - [ ] Pointer/pen/touch handling and palm rejection.
```

Linii importante: 226-300

```
 226 | **Done when:** two devices draw on the same canvas and strokes appear on the other side in < ~300 ms; offline
strokes sync after reconnect; undo only affects your own strokes.
 227 |
 228 | ### Phase 6: Auth, workspaces, onboarding, search (2-3 days)
 229 | - [ ] Supabase Auth: login, register, recovery, error states.
 230 | - [ ] Onboarding: choose workspace purpose (Personal / University / Project), create first notebook.
 231 | - [ ] Global search (Ctrl K): keyword search first (Postgres full-text); semantic results added in Phase 7.
 232 |
 233 | **Done when:** a new user can register, finish onboarding, and land on an empty notebook state.
 234 |
 235 | ### Phase 7: AI Co-pilot (4-5 days)  **(thesis core)**
 236 | - [ ] Server job: on debounced page save, extract text by block, chunk, embed, upsert `page_chunks`.
 237 | - [ ] `POST /ai/ask`: embed the question, retrieve top-k chunks (filtered by scope: notebook / all), and
answer **only from the retrieved notes**, returning `[n]` citations with page id + quote.
 238 | - [ ] Co-pilot panel: tabs (Ask my notes / To-Do / Summary), scope selector, streaming response, source cards
that link to the page and block, copy, thumbs, "no relevant notes found" state, error state.
 239 | - [ ] Smart tasks: extract tasks + due dates from page text (structured output) into `tasks`; "My tasks"
view.
 240 | - [ ] Handwriting to text and describe sketch (section 6).
 241 | - [ ] Semantic search results in Ctrl K.
 242 |
 243 | **Done when:** a question about your notes returns an answer with clickable citations; a question with no
matching notes says so instead of inventing an answer. Evaluate with a small test set (e.g. 20 questions) and report
accuracy in the thesis.
 244 |
 245 | ### Phase 8: Knowledge graph (2-3 days)
 246 | - [ ] Edges from explicit page links plus embedding similarity above a threshold (`page_links`, kind `ai`).
 247 | - [ ] Force-directed view with filters (tag, date), node preview panel, "Open page", zoom/fit.
 248 | - [ ] Accessible list alternative ("Show accessible list") and an empty state.
 249 |
 250 | **Done when:** the graph for a seeded notebook matches the design and every edge shows why it exists.
```

````
 251 |
 252 | ### Phase 9: P2P proximity sync (3-4 days)  **(thesis core)**
 253 | - [ ] Settings toggle "Offline / P2P"; Proximity Sync modal: scanning, found devices, connecting, connected, sync progress, success, failed, retry.
 254 | - [ ] `y-webrtc` provider over a local signaling server (one device hosts it) or QR/copy-paste connection data.
 255 | - [ ] Merge test: two devices with no internet exchange changes and converge.
 256 |
 257 | **Done when:** two laptops on the same Wi-Fi (internet disabled) sync a page and a drawing, and the demo is repeatable.
 258 |
 259 | ### Phase 10: Settings and export (1-2 days)
 260 | - [ ] Account, sync preferences, AI preferences, export to Markdown/PDF.
 261 |
 262 | ### Phase 11: Responsive, accessibility, polish (3-4 days)
 263 | - [ ] Tablet (stylus workspace, palette docked left/bottom) and mobile (bottom-sheet palette).
 264 | - [ ] Focus rings, keyboard navigation, 44 px touch targets, contrast AA, reduced motion.
 265 | - [ ] Empty/loading/error states everywhere; Dark mode.
 266 | - [ ] Optional: Tauri desktop wrapper.
 267 |
 268 | ### Phase 12: Thesis evaluation (ongoing)
 269 | - [ ] Sync latency and convergence benchmarks.
 270 | - [ ] Offline merge scenarios and results.
 271 | - [ ] RAG quality test set and results.
 272 | - [ ] Short usability test (3-5 people) with a task list.
 273 |
 274 | **Suggested timeline:** phases 0-5 are the core demo. If time is short, **cut phases 8, 10 and Tauri**, and keep 4, 5, 7, 9.
 275 |
 276 | ---
 277 |
 278 | ## 8. Handing the design to Claude (Figma to Claude Code)
 279 |
 280 | ### Option A (recommended): Figma MCP server
 281 | This lets Claude read your actual frames (structure, styles, variables, screenshots).
 282 |
 283 | 1. **Check access.** The Figma MCP server needs your Figma account. Free/Starter plans have a small monthly call limit; a Dev or Full seat on a paid or education plan has far higher limits. Check the current limits at help.figma.com (they change), and if your faculty gave you an education plan, use that account.
 284 | 2. **Add the server to Claude Code** (works the same from the terminal and from the VS Code / JetBrains extension, since they share the configuration):
 285 |    ```bash
 286 |    claude mcp add --transport http figma https://mcp.figma.com/mcp
 287 |    ```
 288 |    Then, inside Claude Code, run `/mcp`, select **figma**, and authenticate in the browser.
 289 |    *(If your Figma version offers the desktop Dev Mode MCP server instead, enable it in the Figma desktop app preferences and use the URL it shows. Follow the current Figma docs if these steps differ.)*
 290 | 3. **Give Claude one frame at a time.** In Figma: right-click the frame → **Copy link to selection**. Then in Claude Code:
 291 |    > Implement the "Workspace with editor" frame from this Figma link: `<paste link>`. Use the React +
````

```
Tailwind setup in this repo and the tokens in `packages/tokens`. Reuse components from `packages/ui`. Match spacing, type and colors exactly. Do not copy absolute positioning.
 292 | 4. **Typical tools Claude will call:** `get_design_context` (structure + code hints), `get_screenshot` (visual reference), `get_variable_defs` (tokens), `get_metadata` (frame tree). If Claude doesn't call them, ask it to.
 293 | 5. **Loop for each screen:** implement → open it in the browser → compare with `get_screenshot` → fix differences. Ask Claude to take a screenshot of your running app (Playwright) and compare.
 294 |
 295 | **Tips**
 296 | - One screen or one component per prompt. Large frames (the whole "Drawing toolkit" sheet) should be split by section: "the Desktop bottom palette only".
 297 | - Start with tokens (Phase 1), then base components, then screens. Doing screens first leads to hard-coded colors.
 298 | - Because node IDs change, always paste a **fresh link**.
 299 | - Tell Claude which Figma details are *not* final (icons, hard-coded fills, missing sidebar icons, section 4) so it doesn't copy mistakes.
 300 |
```

5. Cum explici modificările ca arhitectură

Fluxul actual: browserul pornește în `App.tsx`, citește starea din `appStore.ts`, redirecționează către auth/onboarding și apoi montează `AppShell`. Shell-ul afișează Sidebar și rutele funcționale. EditorPage compune RichEditor și CanvasBlock, iar CopilotPanel/SearchDialog sunt panouri controlate din store.

Persistența actuală: middleware-ul Zustand serializează subsetul de state în localStorage sub cheia `second-brain-app`. Este util pentru demo offline local, dar nu rezolvă colaborarea între dispozitive și nu oferă securitate.

Limitarea importantă: în sursele actuale nu apar `fetch`, `axios`, server Express/Fastify/Nest, schema SQL, Supabase client, Yjs sau Hocuspocus. Asta înseamnă că nu există încă o graniță frontend/backend implementată.

Ce a făcut AI-ul în această etapă: a generat/structurat componentele UI, tipurile și fluxurile locale, plus documentul de implementare care definește arhitectura țintă. Codul este un prototip de produs și o bază vizuală, nu încă implementarea completă a tezei.

6. Planul pentru următorul milestone

Milestone recomandat: vertical slice Auth + pagină persistentă + sync minimal

1. **Separă serverul:** adaugă `apps/server` cu Node/Fastify sau Hocuspocus și `packages/shared` pentru tipuri comune.
2. **Adaugă Supabase:** configurează Auth și tabelele `workspaces`, `notebooks`, `pages`, `members`; activează Row Level Security.
3. **Înlocuiește demo login:** clientul primește sesiune/JWT, serverul verifică tokenul și fiecare query filtrează după `owner_id`/`member`.
4. **Persistă o pagină:** adaugă API pentru citire/scriere și salvează conținutul; apoi migrează editorul către un `Y.Doc` per pagină.
5. **Testează măsurabil:** build + lint, test de auth, două tab-uri care editează aceeași pagină și verifică faptul că stările converg.

Definition of Done: un utilizator se poate autentifica, vede doar workspace-ul propriu, creează/deschide o pagină, editează conținutul, reîncarcă pagina fără pierderi și două tab-uri ajung la același text.

7. Pașii ulteriori către teza completă

- **Phase 4:** Hocuspocus + Yjs + y-indexeddb, status synced/syncing/offline și test Playwright multi-tab.
- **Phase 5:** mută stroke-urile în Y.Map, awareness pentru live strokes și undo per utilizator.
- **Phase 6:** full-text search în Postgres, auth complet și onboarding persistent.
- **Phase 7:** pipeline RAG: chunking, embeddings, pgvector, top-k retrieval, citări și protecția cheilor AI pe server.
- **Phase 8-11:** graph din relații persistente, settings/export, responsive layouts, accesibilitate și măsurători de latență.

> **Mesaj final pentru coordonator:** „Am livrat fundația frontend și am documentat arhitectura backend. Următorul increment nu este să mai adaug mock-uri, ci să fac prima felie verticală reală: Auth +

> Postgres + pagină persistentă, apoi sincronizare CRDT măsurată.”

Anexă — fișiere relevante

Fișiere analizate: `package.json`, `apps/web/package.json`, `apps/web/src/App.tsx`, `apps/web/src/store/appStore.ts`, toate paginile din `apps/web/src/pages`, toate componentele din `apps/web/src/components`, `apps/web/src/index.css`, `apps/web/src/App.css`, `apps/web/vite.config.ts` și `SECOND_BRAIN_IMPLEMENTATION.md`. Ramura analizată este `v1.0`, nu `main`, deoarece `main` conține doar README-ul inițial.